МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Новосибирский национальный исследовательский государственный
университет»
(Новосибирский государственный университет, НГУ)
Структурное подразделение Новосибирского государственного университета –
Высший колледж информатики Университета (ВКИ НГУ)
КАФЕДРА ИНФОРМАТИКИ

Веб-приложение для администрирования баз данных

Квалификация программист

ВКР_2207г_Бородина.А.В.

Руководитель Цимбалист Г.Н.
Лаборант МЛабИК СУНЦ НГУ

«___»_____2026 г.

Студент 4 курса Бородина А.В.
гр. 2207 д2 «____»_____2026 г.

Новосибирск

2026

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, УСЛОВНЫХ ОБОЗНАЧЕНИЙ И ТЕРМИНОВ 4
ВВЕДЕНИЕ 5
1 ПОСТАНОВКА ЗАДАЧИ ВКР 6
1.1 Бизнес-требования 6
1.2 Пользовательские требования 7
1.3 Системные требования 7
1.4 Требования к пользовательскому интерфейсу 7
1.5 План-график выполнения ВКР 8
2 АНАЛИЗ ТРЕБОВАНИЙ И ОПРЕДЕЛЕНИЕ СПЕЦИФИКАЦИЙ 9
2.1 Описание предметной области задачи ВКР 9
2.1.1 Информационные объекты предметной области и взаимосвязи между ними 9
2.1.2 Информационные потребности пользователей 10
2.1.3 Методы работы с информационными объектами предметной области 10
2.1.3.1 Способы хранения информации об объектах предметной области 10
2.1.3.2 Математические модели, используемые для обработки информации 11
2.1.3.3 Применяемые программные технологии обработки информации, основанные на математических моделях 11
2.1.3.4 Способы интерпретации и визуального представления информации 11
2.1.3.5 Технологии получения и передачи информации 12
2.1.4 Обзор существующих программных реализаций решения задачи 12
2.1.5 Концептуальное обоснование разработки 12
2.2 Классы и характеристики пользователей 12
2.3 Функциональные требования 13
2.3.1 Определение функциональных возможностей 13
2.3.2 Описание прецедентов 13
2.4 Нефункциональные требования 14
3 ВЫБОР ПРОГРАММНЫХ СРЕД И СРЕДСТВ РАЗРАБОТКИ 15
3.1 Сравнительный анализ имеющихся возможностей по выбору средств разработки 15
3.2 Характеристика выбранных программных сред и средств 15
4 АЛГОРИТМЫ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ 16
4.1 Этапы реализации 16
4.2 Пользовательский интерфейс 16
4.2.1 Описание взаимодействия с пользователем 16
4.2.2 Определение операций пользователей и составление функциональных блоков 16
4.2.3 Проектирование структуры экранов и схемы навигации 17

4.2.4 Разработка дизайна интерфейса 17
4.3 Входные, выходные и промежуточные данные 18
4.4 Реализация используемых методов хранения, обработки и передачи информации об объектах предметной области 18
4.4.1 Методы хранения данных 18
4.4.2 Алгоритмы реализации используемых математических моделей 19
4.4.3 Алгоритмы использования применяемых программных технологий обработки данных 20
4.4.4 Алгоритмы применения методов графического анализа данных 20
4.4.5 Алгоритмы использования технологий передачи данных 20
4.5 Описание архитектурного решения 21
4.5.1 Структурная организация программной системы 21
4.5.2 Архитектура программного кода 21
5 ТЕСТИРОВАНИЕ И ОПТИМИЗАЦИЯ 22
5.1 План тестирования 22
5.2 Результаты тестирования и оптимизация 22
6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ 23
ЗАКЛЮЧЕНИЕ 24
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 25
ПРИЛОЖЕНИЯ 26
Приложение А 26
Приложение Б 26
Приложение В 26
Приложение Г 26
Приложение Д 26
Приложение Е 27

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, УСЛОВНЫХ ОБОЗНАЧЕНИЙ И ТЕРМИНОВ

БД (База данных) — организованное хранилище информации.

СУБД (Система управления базами данных) — программное обеспечение, предназначенное для создания, ведения, защиты и обеспечения доступа к базам данных; в проекте используется Microsoft SQL Server как основная СУБД.

VHD (Virtual Hard Disk) — формат виртуального жёсткого диска, применяемый в проекте для хранения данных базы как целостного файла.

Клон — рабочая, независимая копия базы данных, развёрнутая на основе VHD-образа; подключается к SQL Server как полноценная база.

Образ — исходная, неизменяемая VHD-копия базы данных, созданная из резервной копии (бэкапа) или существующей базы; используется в качестве шаблона для создания клонов.

Веб-приложение — программное приложение, доступное через веб-браузер по сети, состоящее из клиентской (frontend) и серверной (backend) частей, обеспечивающее пользователям работу с данными без необходимости установки на устройство.

Backend — серверная часть веб-приложения, реализующая бизнес-логику, обработку данных и взаимодействие с базой данных.

Frontend — клиентская часть веб-приложения, отображающая интерфейс в браузере и обеспечивающая интерактивное взаимодействие пользователя с системой.

API (Application Programming Interface) — интерфейс программирования приложений, обеспечивающий обмен данными между клиентской и серверной частями веб-приложения, а также интеграцию с внешними сервисами.

C# (C-Sharp) — объектно-ориентированный язык программирования.

Framework (EF) — объектно-реляционный фреймворк для платформы .NET, используемый для работы с базой данных через объекты языка C# для разработки серверной части веб-приложения.

Blazor — фреймворк от Microsoft для создания интерактивных веб-приложений с использованием C#. В проекте используется версия Blazor Server для реализации клиентской части.

HTML и CSS — стандартные веб-технологии, совместно определяющие структуру и визуальное оформление пользовательского интерфейса веб-приложения.

ВВЕДЕНИЕ

В современных корпоративных информационных системах базы данных Microsoft SQL Server часто достигают значительных объёмов — от 1 ТБ и выше. Такие размеры характерны для бизнес-приложений, обслуживающих учёт, логистику, финансы и другие критически важные процессы. При разработке, тестировании и отладке новых функций разработчикам регулярно требуется работать с актуальными, но изолированными копиями рабочих баз данных. Однако стандартные методы клонирования в условиях больших объёмов данных могут занимать несколько часов, что неприемлемо.

В связи с этим возникла практическая потребность в разработке собственного корпоративного инструмента, который бы:

- Автоматизировал процесс клонирования через веб-интерфейс.
- Использовал технологию VHD-образов на основе Hyper-V для быстрого развёртывания.
- Обеспечивал централизованное управление и мониторинг всех клонов и образов;
- Интегрировался в существующую Windows-инфраструктуру без необходимости внешних зависимостей.

Целью данной выпускной квалификационной работы является разработка веб-приложения для централизованного управления копиями баз данных MS SQL, позволяющего администраторам и разработчикам быстро создавать, удалять и отслеживать клонированные БД через интуитивный интерфейс.

Для достижения цели были поставлены следующие задачи:

- Разработать архитектуру программного средства, включающую веб-клиент, серверную часть и агента на SQL-серверах.
- Реализовать функционал создания VHD-образов и развёртывания клонов на их основе.
- Обеспечить управление через веб-интерфейс с поддержкой мониторинга используемых образов и клонов.
- Провести тестирование и отладку ПО в условиях, приближенных к реальной эксплуатации.

Объектом исследования выступает процесс клонирования баз данных Microsoft SQL Server в корпоративной среде. Предметом — программное средство, реализующее автоматизацию этого процесса на основе виртуализации и веб-технологий.

Разработанное решение имеет высокую практическую значимость: оно позволяет сократить время создания клона с нескольких часов до нескольких минут, исключить ошибки, связанные с ручными операциями, и обеспечить полный контроль над использованием ресурсов.

1 ПОСТАНОВКА ЗАДАЧИ ВКР

1.1 Бизнес-требования

От программного средства ожидается повышение эффективности работы за счёт автоматизации процессов клонирования баз данных, сокращение временных затрат разработчиков и администраторов, минимизация рисков ошибок при ручном управлении копиями и обеспечение централизованного контроля над использованием дисковых ресурсов. Основной фокус направлен на ускорение подготовки изолированных тестовых сред, упрощение взаимодействия между разработчиками, а также обеспечение прозрачности и аудируемости всех операций с образами и клонами баз данных. От ПС также ожидается:

1. Повышение эффективности работы IT-отдела за счёт сокращения времени создания копий баз данных с нескольких часов до нескольких минут.
2. Обеспечение централизованного и безопасного управления копиями баз данных через единый веб-интерфейс.
3. Снижение зависимости от ручных операций и минимизация ошибок.
4. Создание возможности контроля за использованием дискового пространства и предотвращения конфликтов баз и VHD-файлов.
5. Обеспечение соответствия корпоративным требованиям к безопасности и локализации данных.
6. Повышение качества и скорости разработки программного обеспечения за счёт быстрого предоставления актуальных, изолированных копий баз данных для тестирования и отладки.

1.2 Пользовательские требования

Пользовательские требования отражают основные ожидания и сценарии взаимодействия пользователей с веб-приложением для управления копиями баз данных Microsoft SQL Server. Они включают функциональность для создания и удаления образов и клонов, мониторинга состояния копий и безопасного управления ресурсами через централизованный интерфейс.

1. Авторизация пользователей с использованием логина и пароля (роли не разделены, доступ предоставляется разработчикам баз данных, системным администраторам).

2. Выбор целевого SQL-сервера из списка развёрнутых агентов для выполнения операций.
3. Создание VHD-образа базы данных либо из резервной копии (.bak), либо из существующей подключённой базы.
4. Создание клонов на основе ранее созданных VHD-образов с указанием имени и пути размещения.
5. Удаление как отдельных клонов, так и исходных образов.
6. Просмотр актуальной информации о всех образах и клонах на выбранном сервере.
7. Подтверждение критических операций (например, удаления) через диалоговые окна.
8. Отображение результата выполнения операций («Успех» / «Ошибка» с описанием) в интерфейсе.
9. Индикация выполнения длительных операций («Загрузка…») для предотвращения повторных запросов.
10. Интуитивно понятный и единообразный веб-интерфейс с поддержкой современных браузеров (Chrome, Edge).

1.3 Системные требования

Приложение построено по клиент-серверной архитектуре и включает три основные части:

- **Веб-клиент**: Централизованная пользовательская оболочка, предоставляющая интерфейс для запуска операций клонирования, удаления и мониторинга.
- **Бэкенд**: Серверная часть приложения, отвечающая за маршрутизацию запросов от веб-клиента, пересылку команд к соответствующему агенту и передачу результатов операций обратно.
- **Агент**: Фоновая служба Windows, развёрнутая на каждом отслеживаемом SQL-сервере. Реализует всю бизнес-логику: восстановление баз из .bak, создание/удаление VHD-файлов, подключение/отключение баз к СУБД, сбор данных для мониторинга. Работает автономно, настраивается через конфигурационный файл, поддерживает запуск и остановку через стандартные средства Windows.
- **Хостинг**: развёртывание на серверах компании. Серверная часть и веб-клиент размещаются на выделенном сервере. Агенты устанавливаются непосредственно на серверы с MS SQL Server и требуют включённой роли Hyper-V для работы с VHD.

Минимальные требования:

- **Сервер (для размещения бэкенда и веб-клиента) и SQL-сервер (для агента):**
  - ОС: Windows Server 2012 R2 с включённой ролью Hyper-V
  - .NET 6 Runtime (x64) и .NET 6 Hosting Bundle
  - Процессор: Intel Xeon / AMD EPYC
  - ОЗУ: от 1 ГБ
  - Доступ к MS SQL Server с полными правами
  - Диск: 200 МБ свободного места
- **Клиент:**
  - Современный браузер: Chrome, Edge (поддержка HTML5, JavaScript)
  - Доступ к внутреннему IP-адресу сервера приложения

1.4 Требования к пользовательскому интерфейсу

Дизайн веб-приложения должен быть лаконичным, функциональным и ориентированным на задачи, без избыточной графики. Интерфейс реализуется как веб-приложение, доступное через браузер, и не требует адаптации под мобильные устройства, так как основное использование предполагается со стационарных устройств.

Требования к внешнему виду и формам взаимодействия:

- Стиль интерфейса — минималистичный. С акцентом на элементы управления(кнопки).
- Цветовая палитра строго соответствует корпоративным требованиям:
  - Основной цвет: #0c54a0 (темно-синий) — используется для заголовков, активных элементов меню, кнопок действий.
  - Акцентный цвет: #ef7f1a (оранжевый) — применяется для предупреждений, важных уведомлений и выделения статусов.
  - Фон и текст: #ffffff (белый) — обеспечивает высокую читаемость.
- Шрифт — единый для всего приложения: современный, моноширинный или без засечек, размер 14–16 px.
- Структура интерфейса — фиксированная, с боковой панелью навигации и верхней панелью выбора сервера.

Требования к размещению элементов управления:

- Header (верхняя панель):
  - В правом верхнем углу — выпадающий список «SQL сервер» для выбора целевого сервера.

    - В правом углу — имя авторизованного пользователя.
- Sidebar (боковое меню):
    - Расположено слева, постоянно открыто.
    - Содержит пункты: Управление образами, Управление клонами, Мониторинг.
    - Текущая страница выделяется цветом #0c54a0.
- Основная область:
    - Содержит заголовок раздела, кнопку действия («Создать образ» / «Создать клон»), таблицу с данными (образами или клонами) и форму создания (при переходе по кнопке).
    - В таблицах каждая запись содержит кнопку «Удалить».

Требования к сообщениям и обратной связи:

- Все критические действия (удаление образа/клона) требуют подтверждения во всплывающем окне с заголовком «ВНИМАНИЕ» и кнопками «ОК» / «Отмена».
- Результат операции отображается в диалоговом окне:
    - «РЕЗУЛЬТАТ – УСПЕХ!» — при успешном завершении (кнопка «ОК»).
    - «РЕЗУЛЬТАТ – ОШИБКА!» — при сбое (кнопка «ОК» и ссылка «см. подробности» с расшифровкой ошибки).
- При выполнении длительных операций (создание образа, клонирование) на странице отображается индикатор «Загрузка…», блокирующий повторные нажатия.

Требования к форматам ввода:

- Поля ввода:
    - «Наименование» — текстовая строка без спецсимволов, уникальная в пределах сервера.
    - «Путь» — абсолютный путь к директории (например, D:\MySQLClone\).
    - Путь к .bak-файлу (при создании из бэкапа) — полный путь к файлу с расширением .bak.
- Выбор источника создания — переключатель между вариантами:
    - «Создать образ из бэкапа» → появляется поле для ввода пути к .bak;
    - «Создать образ из существующей БД» → появляется выпадающий список баз данных, полученный от агента.

Доступ к функциональности:

- Вся функциональность доступна только через графический интерфейс — нет командной строки или API для конечного пользователя.
- Навигация не предусматривает глубоких вложенных страниц — все ключевые действия выполняются в рамках трёх основных разделов.

- После авторизации пользователь сразу попадает на страницу «Управление образами» — это главная рабочая область.

1.5 План-график выполнения ВКР

В таблице представлен план с указанием дат работы над ВКР.

Таблица 1 - План выполнения ВКР

№	Этап	Описание	Дата начала	Дата конца	Результат	Контрольная точка
1	Анализ предметной области и требований пользователей	Изучение процессов клонирования баз данных, выявление проблем ручного управления, формирование функциональных и нефункциональных требований на основе потребностей администраторов и разработчиков.	10.01.26	19.01.26	Список пользовательских и системных требований	Защита отчёта по анализу требований
2	Проектирование архитектуры веб-приложения	Определение технологического стека (.NET 6, Blazor Server), проектирование трёхкомпонентной архитектуры (Web-клиент, Backend, Агент), выбор способа взаимодействия между компонентами.	20.01.26	30.01.26	Архитектурная схема, диаграммы UML (прецедентов, последовательности), техническое задание	Утверждение архитектуры системы
3	Разработка библиотеки агента	Реализация классов и сервисов для работы с VHD, восстановления из .bak, подключения/отключения БД в MS SQL.	31.01.26	28.02.26	Библиотека с полным набором методов: создание/удаление образов и клонов, мониторинг	Демонстрация работы через консольное приложение или unit-тесты
4	Разработка пользовательского интерфейса (UI)	Создание макетов страниц: авторизация, управление образами,	01.03.26	06.03.26	Прототипы интерфейсов, дизайн-система,	Презентация дизайна и сбор отзывов

		управление клонами, мониторинг; реализация единого дизайна.			HTML/CSS-шаблоны	
5	Реализация модуля авторизации и навигации	Разработка страницы входа, бокового меню, выпадающего списка выбора SQL-сервера, общей структуры Blazor-приложения.	07.03.26	31.03.26	Рабочий UI-фреймворк с авторизацией и навигацией	Тестирование корректности переходов и отображения элементов
6	Разработка серверной части (Backend)	Реализация ASP.NET Core API для приёма запросов от фронтенда, маршрутизации команд к агенту, обработки результатов операций.	01.04.26	15.04.26	Рабочий Backend с контроллерами для всех операций	Проверка корректности визуализации данных
7	Интеграция компонентов и реализация мониторинга	Объединение Web-клиента, Backend и Агента; реализация страницы «Мониторинг» с отображением списков образов и клонов.	16.04.26	30.04.26	Полностью интегрированное приложение с рабочим мониторингом	Демонстрация сквозного сценария: создание. мониторинг, удаление
8	Тестирование и отладка системы	Проведение функционального тестирования всех сценариев (создание/удаление образов и клонов, проверка ошибок, валидация путей и имён), исправление дефектов.	31.04.26	15.05.26	Тестовые отчёты и стабильная сборка приложения	Подтверждение готовности к развёртыванию
9	Развёртывание и внедрение приложения	Настройка сервера (установка .NET 6 Hosting Bundle, IIS), деплой Web-клиента и Backend, установка агентов на тестовые SQL-серверы, подготовка документации и демонстрационного стенда.	16.05.26	31.05.26	Рабочая версия веб-приложения	Итоговый отчёт и демонстрация проекта

2 АНАЛИЗ ТРЕБОВАНИЙ И ОПРЕДЕЛЕНИЕ СПЕЦИФИКАЦИЙ

2.1 Описание предметной области задачи ВКР

2.1.1 Информационные объекты предметной области и взаимосвязи между ними

Веб-приложение для централизованного управления копиями баз данных Microsoft SQL Server разрабатывается для IT-отдела ООО «БАЙТ» с целью автоматизации рутинных операций, связанных с подготовкой изолированных тестовых и разработческих сред. Система обеспечивает быстрое создание, удаление клонов рабочих баз данных на основе технологии VHD-образов, что позволяет сократить время развёртывания с нескольких часов до нескольких минут.

Основная цель — исключить ручные операции по клонированию баз данных, восстановлению из бэкапов, копированию файлов и управлению дисковым пространством, заменив их единым, контролируемым и аудируемым процессом через веб-интерфейс.

Автоматизируемые операции:

1. Создание образа — формирование неизменяемого шаблона на основе существующей базы данных или резервной копии (.bak). Образ используется как основа для последующего клонирования.
2. Создание клона — развёртывание рабочей копии базы данных на основе ранее созданного образа. Клон полностью функционален и изолирован от оригинала.
3. Удаление клона — безопасное удаление рабочей копии базы данных с освобождением занятого дискового пространства.
4. Удаление образа — удаление шаблона при условии отсутствия зависимых от него клонов.
5. Мониторинг состояния — сбор и отображение актуальной информации о всех доступных образах и клонах на выбранном сервере.

На основе анализа предметной области выделяются следующие ключевые информационные объекты:

1. Образ (Image) — неизменяемый шаблон базы данных, созданный для целей клонирования. Характеризуется именем, путём хранения, размером и датой создания.

2. Клон (Clone) — рабочая копия базы данных, развёрнутая на основе образа. Характеризуется уникальным именем, путём хранения, размером, ссылкой на исходный образ и текущим состоянием.
3. SQL-сервер — целевой сервер, на котором выполняются операции с базами данных. Является контекстом для всех действий с образами и клонами.
4. Операция — зафиксированное действие пользователя (создание, удаление, запрос мониторинга), включающее тип, целевой объект, результат и временную метку. Используется для аудита и контроля.
5. Пользователь — сотрудник IT-отдела, отвечающих за подготовку и управление изолированных копий баз данных Microsoft SQL Server и имеющий доступ к функционалу системы.

Эти объекты находятся в строгих взаимосвязях:

- Один образ может породить множество клонов
- Каждый клон зависит ровно от одного образа
- Каждая операция фиксирует изменение состояния одного из объектов.

2.1.2 Информационные потребности пользователей

Пользователи системы нуждаются в быстром, надёжном и прозрачном доступе к информации о состоянии тестовых и разработческих баз данных. Им важно не просто выполнять операции, а понимать, что происходит с системой в каждый момент времени.

Вот что конкретно нужно пользователям:

- Знать, какие образы существуют, чтобы не создавать дубликаты и выбирать правильный образ для клонирования.
- Видеть список всех клонов, чтобы понимать, какие тестовые среды сейчас развёрнуты, сколько места они занимают и на каком сервере работают.
- Понимать, от какого образа создан каждый клон, чтобы не удалить образ, от которого зависят активные клоны.
- Получать чёткий результат после каждой операции. Успешно ли прошло создание/удаление, или произошла ошибка (и почему).
- Иметь возможность быстро найти нужный образ или клон, особенно когда их становится много.
- Контролировать использование дискового пространства, чтобы не заполнить диск случайно и вовремя удалять ненужные копии.
- Убедиться, что операция выполнена именно там, где нужно ведь у компании несколько SQL-серверов.

Все эти данные пользователи получают через единый веб-интерфейс:

- в виде таблиц (образы, клоны),
- с отображением размера и пути,
- с подтверждением критических действий,
- и с понятными сообщениями об ошибках.

Таким образом, система устраняет неопределённость и позволяет принимать решения на основе актуальной информации.

2.1.3 Методы работы с информационными объектами предметной области

2.1.3.1 Способы хранения информации об объектах предметной области

Для надёжного, структурированного и масштабируемого хранения данных используется реляционная база данных (БД). Выбор реляционной модели обусловлен чёткой структурой предметной области: сущности (образы и клоны) имеют фиксированные атрибуты и устойчивые связи между собой. БД обеспечивает целостность данных, поддержку транзакций и возможность аудита всех операций.

- БД содержит следующие ключевые таблицы:
- Images — данные об образах (идентификатор, имя образа, путь к VHD-файлу, имя исходной базы данных, имя файла VHD, корневой путь к каталогу, дата создания, дата удаления);
- Clones — данные о клонах (идентификатор, имя клона, имя образа, ссылка на исходный образ, путь к исходному VHD-образу, путь к VHD-файлу клона, корневой путь к каталогу, дата создания, дата удаления).

Хранение файлов VHD осуществляется следующим образом:

- Файловая система: Все VHD-файлы хранятся в определённых каталогах на выбранном сервере.

2.1.3.2 Математические модели, используемые для обработки информации

В рамках задачи управления копиями баз данных не применяются сложные математические или статистические модели. Предметная область носит операционно-административный характер и не требует вычислений, прогнозирования или оптимизации.

Однако для обеспечения корректности операций используются логические правила и ограничения, формализуемые как:

- Правило уникальности имени клона:

$$\forall c \in \text{Clones:name}(c) \notin \{\text{name}(d) \mid d \in \text{Databases}\}$$

где Databases— множество активных баз данных на сервере.

- Правило зависимости образа:

$$\text{DeleteImage}(i) \Rightarrow \nexists c \in \text{Clones:source}(c) = i$$

то есть образ может быть удалён только при отсутствии зависимых клонов.

- Правило контроля дискового пространства:

$$\text{CreateVHD}(\text{size}) \Rightarrow \text{FreeSpace}(\text{disk}) \geq \text{size}$$

Эти правила реализуются в виде условных операторов (if) в коде агента и обеспечивают целостность и безопасность операций.

2.1.3.3 Способы интерпретации и визуального представления информации

Для представления информации пользователю используются табличные и текстовые формы, что соответствует характеру задачи — оперативное управление и контроль.

Основные методы визуализации:

- Таблицы данных — основной способ отображения информации об объектах.
- Цветовое кодирование статусов: зелёный — успешная операция, красный — ошибка.
- Диалоговые окна — для подтверждения критических действий (удаление) и отображения результатов.

Графики, диаграммы и сложные визуализации не используются, так как они не нужны для решения задач.

2.1.3.4 Технологии получения и передачи информации

Обмен данными в системе организован по трёхзвенной архитектуре:

1. Веб-клиент (Blazor WebAssembly) взаимодействует с Бэкенд (ASP.NET Core): Общение осуществляется через HTTP-запросы к контроллерам бэкенда. Все данные передаются в формате JSON.
2. Бэкенд взаимодействует с Агентом: Агент запускается на каждом SQL-сервере и периодически опрашивается бэкендом через REST API (например, GET /api/agent/status). При выполнении операции (создание, удаление) бэкенд отправляет команду в формате JSON.

3. Агент взаимодействует с SQL Server и файловой системой: Агент взаимодействует с SQL Server через T-SQL-запросы (SELECT, RESTORE, CREATE DATABASE), а с файловой системой — через стандартные .NET-методы (File.Copy, Directory.Exists). Все соединения защищены через HTTPS.

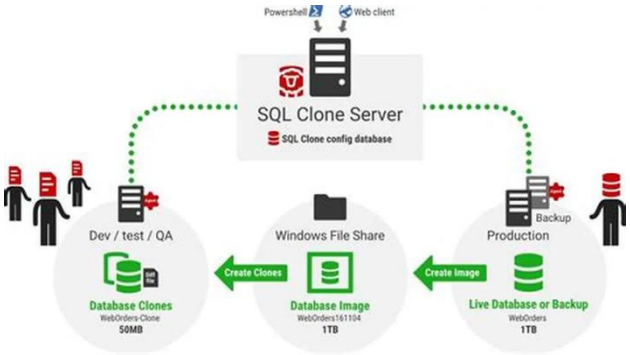
2.1.5 Обзор существующих программных реализаций решения задачи

Среди существующих решений можно выделить следующие аналоги:

1. Microsoft SQL Server Management Studio (SSMS)
   - Достоинства: встроенные средства резервного копирования, восстановления, скриптов.
   - Недостатки: полностью ручной процесс, отсутствие централизованного управления, нет поддержки VHD, высокая трудоёмкость при работе с большими БД.

Изображение 1 - Microsoft SQL Server Management Studio

1. Redgate SQL Clone
   - Достоинства: автоматизация клонирования через образы, интеграция с DevOps.
   - Недостатки: коммерческое решение, высокая стоимость лицензий, требует установки дополнительного ПО, не использует VHD, ориентировано на облачные среды.



Изображение 2 - Redgate SQL Clone

Сравнительный анализ показывает, что ни одно из решений не сочетает:

- использование VHD-технологии (встроенной в Windows),
- централизованный веб-интерфейс,
- минимальную зависимость от внешних компонентов,
- автоматизацию без лицензионных затрат.

Разрабатываемое приложение обладает практической новизной за счёт веб-управления и локальной архитектуры, ориентированной на корпоративную Windows-инфраструктуру.

2.1.5 Концептуальное обоснование разработки

Современные подходы к клонированию баз данных либо слишком дороги, либо использует неавтоматизированный подход. Это создаёт операционное узкое место в процессах тестирования и разработки: подготовка среды занимает часы, а ошибки при ручном копировании приводят к потерям времени и данных.

Разработка нового решения обоснована необходимостью автоматизировать именно процесс подготовки изолированных копий в условиях стандартной Windows-инфраструктуры.

Эта концепция:

- снижает стоимость внедрения до нуля (только внутренняя разработка),
- ускоряет клонирование до нескольких минут,
- обеспечивает полный контроль,

Такой подход не реализован в существующих коммерческих решениях и представляет собой оригинальное инженерное решение для корпоративной среды.

2.2 Классы и характеристики пользователей

Разрабатываемое веб-приложение предназначено для сотрудников IT-отдела, участвующих в подготовке и управлении тестовыми и разработческими средами на основе копий рабочих баз данных Microsoft SQL Server. Все пользователи обладают единым уровнем доступа и технической квалификацией, достаточной для выполнения административных операций с СУБД и файловой системой.

Подробные характеристики пользовательской роли приведены в таблице 2.

Таблица 2 – Описание пользовательских ролей

Пользователь	IT-специалист (администратор баз данных / разработчик баз данных)
Социальные характеристики	Сотрудник IT-отдела компании; имеет высшее или среднее профессиональное образование в области информационных технологий; работает в команде, взаимодействует с разработчиками и тестировщиками.
Мотивационная среда	Производственная необходимость: сокращение времени подготовки тестовых сред, минимизация рисков при ручном клонировании, централизованный контроль над использованием ресурсов.
Навыки и умения	Знание работы с Microsoft SQL Server (восстановление БД, управление файлами .mdf/.ldf), базовые навыки работы с Windows Server.
Требования к ПО	Возможность создания VHD-образов и клонов через веб-интерфейс; поддержка выбора целевого SQL-сервера; отображение актуального состояния всех копий; подтверждение критических действий; индикация выполнения длительных операций.
Задачи пользователя	• Создание VHD-образов из существующих БД или .bak-файлов • Создание клонов на основе образов • Удаление клонов и образов с проверкой зависимостей • Мониторинг состояния всех копий (имя, путь, размер, источник) • Контроль использования дискового пространства
Рабочая среда	Рабочая станция с доступом к внутренней сети компании; современный браузер (Chrome/Edge); серверы с установленным Microsoft SQL Server и ролью Hyper-V; права администратора на целевых серверах для запуска агента.

2.3 Функциональные требования

2.3.1 Определение функциональных возможностей

На основе анализа предметной области можно выделить функциональность приложения, исходя из целей и задач пользователя. Перечень функций, необходимых в приложении для пользовательского профиля IT-специалист:

- создание VHD-образа на основе существующей базы данных или резервной копии (.bak);
- создание клона базы данных на основе ранее созданного VHD-образа;
- удаление клона базы данных с освобождением дискового пространства;
- удаление VHD-образа с проверкой отсутствия зависимых клонов;
- выбор целевого SQL-сервера из списка развёрнутых агентов;

- просмотр списка доступных образов, клонов;
- мониторинг состояния всех копий на выбранном сервере;
- подтверждение критических операций (удаление) через диалоговое окно;
- получение результата выполнения операции и индикация выполнения длительных операций.

На основе анализа предметной области и пользовательских требований к разрабатываемой информационной системе составлен реестр юзкейсов для выделенной пользовательской роли (см. таблица 3).

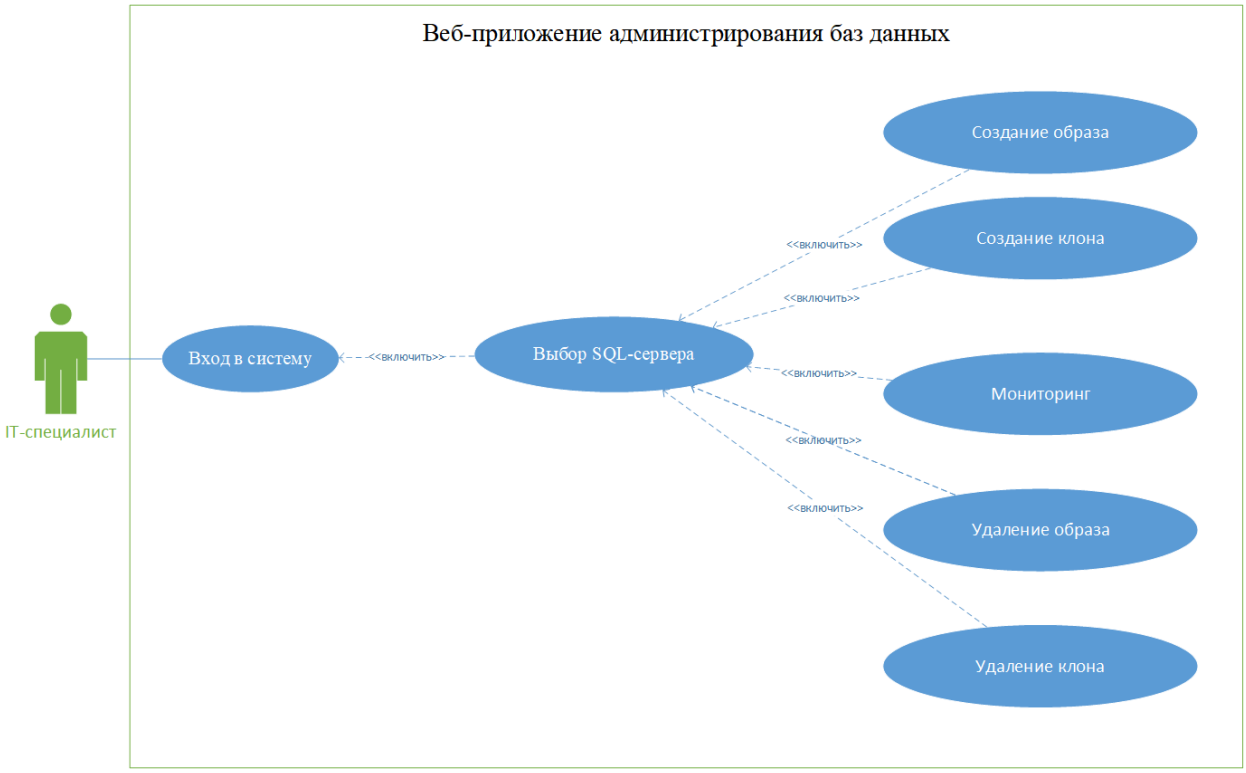
Таблица 3 – Реестр юскейсов

Пользователь	**Потребность пользователя**	**Use case**	**Пользовательская история**	**Сценарий использования**
IT-специалист	Создать шаблон(образ) для последующего клонирования	Создание образа	IT-специалист нажимает на кнопку «Создать образ». Открывается форма, в которой он выбирает тип источника («Из существующей БД» или «Из .bak-файла»), указывает имя образа и путь для сохранения VHD-файла. Нажимает «Создать», система выполняет операцию и отображает результат.	• Выбор SQL-сервера из выпадающего списка • Переход в раздел «Управление образами» • Нажатие кнопки «Создать образ» • Заполнение данными • Нажатием на кнопку «Создать» • Получение уведомления о результате
	Развернуть рабочую копию БД для тестирования или разработки	Создание клона	IT-специалист нажимает на кнопку «Создать клон». В открывшейся форме выбирает существующий образ, указывает имя нового клона и путь для VHD-файла. После подтверждения система создаёт клон и подключает его к SQL Server.	• Выбор SQL-сервера • Переход в раздел «Управление клонами» • Нажатие «Создать клон» • Выбор образа из списка • Заполнение данными

				• Нажатие «Создать» • Получение результата операции
	Удалить шаблон(образ), если он больше не нужен	Удаление образа	IT-специалист находит образ в таблице, нажимает «Удалить». Система проверяет, существуют ли клоны, созданные на его основе. Если нет — удаляет VHD-файл. Если есть — блокирует операцию и выводит сообщение.	• Переход в раздел «Управление образами» • Выбор образа в таблице • Нажатие «Удалить» и подтверждение действия • Получение результата
	Освободить дисковое пространство, удалив ненужную копию	Удаление клона	IT-специалист находит нужный клон в таблице, нажимает «Удалить». Система запрашивает подтверждение, отключает БД от SQL Server, удаляет VHD-файл и показывает результат.	• Переход в раздел «Управление клонами» • Поиск нужного клона в таблице • Нажатие «Удалить» и подтверждение во всплывающем окне • Получение результата
	Получить актуальную информацию о состоянии копий	Мониторинг	IT-специалист переходит в раздел «Мониторинг». Система отображает две таблицы: «Образы» (имя, путь, размер) и «Клоны» (имя, путь, размер, исходный образ). Данные обновляются автоматически при выборе сервера.	• Выбор SQL-сервера • Переход в раздел «Мониторин • Просмотр таблицы «Образы» и просмотр таблицы «Клоны» • Анализ использования дискового пространства и

				зависимостей

На изображение 3 представлена UML-диаграмма вариантов использования ИС.



Изображение 3 - UML-диаграмма вариантов использования ИС

2.3.2 Описание прецедентов

Таблица 2.3.2.1 – Описание прецедента «Создание образа»

Поле	Содержание
Название прецедента	Создание образа
Исполнитель	IT-специалист
Цель	Создание неизменяемого VHD-образа на основе существующей базы данных или резервной копии (.bak) для последующего клонирования.
Основной успешный сценарий	IT-специалист нажимает на кнопку «Создать образ». Открывается форма, в которой он выбирает тип источника («Из существующей БД» или «Из .bak-файла»), указывает имя образа и путь для сохранения VHD-файла. Нажимает «Создать», система выполняет операцию и отображает результат.

Таблица 2.3.2.2 – Описание прецедента «Создание клона»

Поле	Содержание

Название прецедента	Создание клона
Исполнитель	IT-специалист
Цель	Развёртывание рабочей копии базы данных на основе ранее созданного VHD-образа для использования в тестировании или разработке.
Основной успешный сценарий	IT-специалист нажимает на кнопку «Создать образ». Открывается форма, в которой он выбирает тип источника («Из существующей БД» или «Из .bak-файла»), указывает имя образа и путь для сохранения VHD-файла. Нажимает «Создать», система выполняет операцию и отображает результат.

Таблица 2.3.2.3 – Описание прецедента «Удаление клона»

Поле	Содержание
Название прецедента	Создание клона
Исполнитель	IT-специалист
Цель	Освобождение дискового пространства путём безопасного удаления ненужной копии базы данных.
Основной успешный сценарий	IT-специалист нажимает на кнопку «Создать образ». Открывается форма, в которой он выбирает тип источника («Из существующей БД» или «Из .bak-файла»), указывает имя образа и путь для сохранения VHD-файла. Нажимает «Создать», система выполняет операцию и отображает результат.

Таблица 2.3.2.4 – Описание прецедента «Удаление образа»

Поле	Содержание
Название прецедента	Создание клона
Исполнитель	IT-специалист
Цель	Удаление VHD-образа, если он больше не используется для клонирования.
Основной успешный сценарий	IT-специалист находит образ в таблице раздела «Управление образами» и нажимает «Удалить». Система проверяет, существуют ли клоны, созданные на его основе. Если таких клонов нет — удаляет VHD-файл и отображает сообщение «Успешно». Если клоны существуют — операция блокируется, выводится сообщение: «Ошибка: существуют зависимые клоны».

Таблица 2.3.2.5 – Описание прецедента «Мониторинг»

Поле	Содержание
Название прецедента	Создание клона
Исполнитель	IT-специалист
Цель	Получение актуальной информации о состоянии всех образов и клонов на выбранном SQL-сервере.

Основной успешный сценарий	IT-специалист выбирает целевой SQL-сервер из выпадающего списка и переходит в раздел «Мониторинг». Система автоматически запрашивает у агента данные и отображает две таблицы: «Образы» и «Клоны». Данные обновляются при каждом входе в раздел.

2.4 Нефункциональные требования

Для веб-приложения были определены следующие группы нефункциональных требований.

1. Производительность и масштабируемость

- Время отклика на пользовательские действия (создание/удаление образа или клона) должно составлять не более 5 секунд для операций, не связанных с копированием данных (например, получение списка серверов, отображение таблиц).
- Операции, связанные с физическим копированием данных (создание VHD-файла), могут занимать от 5 до 15 минут в зависимости от размера БД — в этом случае система должна отображать индикатор выполнения и не блокировать интерфейс полностью.
- Архитектура приложения допускает масштабирование: добавление новых SQL-серверов с агентами не требует изменения кода веб-клиента.

2. Переносимость и совместимость

- Серверная часть (бэкенд и агент) должна работать на Windows Server 2012 R2 и выше с установленной ролью Hyper-V.
- Веб-клиент должен корректно отображаться в современных браузерах:
    - Google Chrome (актуальная версия),
    - Microsoft Edge (актуальная версия),

  Приложение не конфликтует с другими процессами на сервере, так как агент использует только стандартные Windows API.
- Все компоненты системы (веб-клиент, бэкенд, агент) разрабатываются на .NET 6, что обеспечивает совместимость с целевой платформой.

3. Надёжность, доступность, ремонтопригодность

- Система должна быть доступна всё рабочего времени (внутри корпоративной сети).
- Критические сбои (например, падение агента) не должны приводить к потере данных — все операции с файлами и БД выполняются с проверкой и возможностью отката.

- При возникновении ошибки (например, недостаток места на диске) система должна корректно обработать исключение и сообщить пользователю точную причину.
- Агент как Windows-служба поддерживает автоматический перезапуск при сбое.
- Логирование всех операций (успешных и неудачных) ведётся в локальные файлы на стороне агента, что позволяет проводить диагностику без подключения к центральной системе.

4. Безопасность

- Доступ к веб-приложению ограничен только сотрудниками IT-отдела через внутреннюю аутентификацию (логин/пароль или Windows-аутентификация).
- Все операции передаются по HTTPS, что предотвращает перехват команд и данных.
- Строка подключения к SQL Server хранится в зашифрованном виде в конфигурационном файле агента.
- Приложение защищено от SQL-инъекций за счёт использования параметризованных запросов.
- Пользователь не может указать произвольный путь для VHD-файла — система ограничивает запись только в предопределённые каталоги.

5. Локализация

- Приложение полностью локализовано на русский язык.
- Все сообщения, интерфейс, документация и логи отображаются на русском языке.
- Поддержка других языков не требуется, так как система предназначена исключительно для внутреннего использования в российском филиале компании.

6. Удобство использования (юзабилити)

- Интерфейс должен быть интуитивно понятным для IT-специалиста без необходимости обучения.
- Все ключевые действия (создание, удаление, мониторинг) должны быть доступны в рамках трёх основных разделов без глубокой навигации.
- Критические операции (удаление) требуют подтверждения во всплывающем окне.
- Результат любой операции отображается в виде чёткого сообщения: «Успех» или «Ошибка с описанием».
- Интерфейс не содержит избыточной графики, ориентирован на функциональность, соответствует корпоративному стилю.

3 ВЫБОР ПРОГРАММНЫХ СРЕД И СРЕДСТВ РАЗРАБОТКИ

При разработке веб-приложения для централизованного управления копиями баз данных Microsoft SQL Server необходимо обеспечить: высокую надёжность при работе с критически важными данными, простоту развёртывания в существующей Windows-инфраструктуре компании, минимальную зависимость от внешних компонентов, безопасное взаимодействие с SQL Server и файловой системой, удобство сопровождения.

В связи с этим был проведён анализ возможных технологических стеков и инструментов на основе следующих критериев:

- совместимость с Windows Server 2012 R2 и Hyper-V,
- производительность при работе с большими файлами (VHD),
- простота интеграции с Microsoft SQL Server,
- соответствие корпоративным требованиям к безопасности.

3.1 Сравнительный анализ имеющихся возможностей по выбору средств разработки

Для реализации серверной части веб-приложения существуют следующие технологические стеки:

- C# (.NET 5/6) — нативная интеграция с Windows, высокая производительность, прямой доступ к PowerShell и WMI;
- Node.js (Express) — быстрая разработка API на JavaScript, но слабая интеграция с Windows-специфичными функциями;
- Python (FastAPI/Django) — простота и скорость разработки, но отсутствие поддержки Hyper-V и сложности с вызовом системных команд;
- Java (Spring Boot) — надёжность и масштабируемость, но избыточность для задачи и высокий порог входа.

Каждый стек оценивался по критериям: скорость разработки, встроенные возможности, порог входа, производительность, лёгкость интеграции с Windows API и Hyper-V, затраты на ресурсы, масштабируемость. Результаты сравнения представлены в таблице. (см. таблица 4).

Таблица 4 – Сравнительный анализ средств разработки

Критерий	C# (.NET 6)	Node,js (Express)	Python (FastAPI)	Java (Spring Boot)

Скорость разработки	Высокая	Выше среднего	Очень высокая	Ниже среднего
Встроенные возможности	Высокие	Средние	Средние	Очень высокие
Порог входа	Средний	Низкий	Низкий	Высокий
Производительность	Очень высокая	Высокая	Средняя	Очень высокая
Интеграция с внешними API	Простая	Простая	Сложная	Сложная
Затраты на серверные ресурсы	Низкие	Средние	Низкие	Высокие
Масштабируемость	Очень высокая	Высокая	Средняя	Очень высокая

По совокупности параметров C# (.NET 6) продемонстрировал оптимальное сочетание:

- нативной интеграции с Windows и Hyper-V,
- высокой производительности при работе с большими файлами (VHD),
- развитой экосистемы для создания веб-интерфейсов (Blazor),
- минимальных затрат на ресурсы.

Альтернативные решения были не подходят по следующим причинам:

- Node.js (Express) — требует вызова PowerShell через child_process, что снижает надёжность и усложняет обработку ошибок при работе с VHD.
- Python (FastAPI) — хотя и обеспечивает быструю разработку, не предоставляет прямого доступа к Hyper-V, что критично для управления VHD.
- Java (Spring Boot) — избыточен для задачи: высокая сложность развёртывания, необходимость установки JVM, отсутствие преимуществ в Windows-среде.

Таким образом, C# и .NET 6 обеспечивают наилучший баланс между производительностью, надёжностью, интеграцией с инфраструктурой и простотой сопровождения, что критично для успешного внедрения.

3.2 Характеристика выбранных программных сред и средств

Для реализации веб-приложения для централизованного управления копиями баз данных Microsoft SQL Server были выбраны следующие программные средства:

C# (C-Sharp)

- Назначение: реализация бизнес-логики, агента, серверной части и UI (Blazor).

- Достоинства: строгая типизация, высокая производительность, прямой доступ к Windows API, файловой системе, отличная поддержка в Visual Studio, встроенная работа с асинхронностью.
- Недостатки: требует понимания ООП и платформы .NET, выше порог входа по сравнению с динамическими языками.

.NET 6 (LTS)

- Назначение: кроссплатформенная (но в нашем случае — Windows-ориентированная) платформа выполнения.
- Достоинства: долгосрочная поддержка, высокая производительность, встроенные HTTP-клиенты, JSON-сериализация, поддержка Blazor.
- Недостатки: требует установки Runtime на сервер.

Blazor Server

- Назначение: создание интерактивного веб-интерфейса без JavaScript.
- Достоинства: весь код на C#, поддержка реального времени (SignalR), простота разработки форм и таблиц, встроенная маршрутизация.
- Недостатки: требует постоянного соединения с сервером, не подходит для публичных сайтов с высокой нагрузкой (но идеален для внутреннего корпоративного приложения).

Microsoft.Data.SqlClient

- Назначение: взаимодействие с Microsoft SQL Server через T-SQL-запросы.
- Достоинства: официальный драйвер от Microsoft, поддержка параметризованных запросов, асинхронность, безопасность.
- Недостатки: не предоставляет ORM — все запросы пишутся вручную (что в данном случае является преимуществом, так как задача административная).

Visual Studio 2022 Community

- Назначение: основная среда разработки.
- Достоинства: полная поддержка .NET 6, Blazor, отладки служб, IntelliSense, встроенный Git, профилировщик.
- Недостатки: требует значительных системных ресурсов.

RESTful API (ASP.NET Core)

- Назначение: обмен данными между веб-клиентом и бэкендом, а также между бэкендом и агентами.
- Достоинства: простота, стандартность, лёгкость тестирования через Postman.
- Недостатки: требует чёткой архитектуры (маршрутизация, валидация).

Выбранный набор технологий полностью соответствует целям проекта: использует встроенные возможности Windows, минимизирует внешние зависимости, обеспечивает высокую надёжность и производительность.

4 АЛГОРИТМЫ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ

4.1 Этапы реализации

Разработка веб-приложения для управления копиями баз данных Microsoft SQL Server через VHD-образы была реализована в восемь последовательных этапов, что позволило обеспечить системный подход к созданию программного продукта и минимизировать риски на каждом этапе разработки.

Этап 1. Анализ требований и проектирование архитектуры

Проведён анализ бизнес-требований, изучены аналоги решений, определены функциональные и нефункциональные требования. Разработана трёхзвенная архитектура: веб-клиент (Blazor Server), серверная часть (ASP.NET Core) и агент. Спроектирована структура базы данных и определены интерфейсы взаимодействия между компонентами.

Этап 2. Разработка библиотеки для работы агента и реализация агента

2.1. Разработана библиотека классов, логику работы с VHD-файлами и метаданными. Реализованы интерфейсы для операций клонирования и управления дисковым пространством.
2.2. Разработан агент в виде службы, обеспечивающий фоновое выполнение операций.
2.3. Реализована обработка запросов от серверной части: создание клонов, удаление образов, проверка целостности.
2.3. Внедрено логирование операций агента для последующего аудита и отладки.

Этап 3. Реализация серверной части (бэкенд)

3.1. Создано серверное приложение на базе ASP.NET Core с настройкой контейнера зависимостей.
3.2. Реализованы контроллеры API для управления сущностями предметной области.
3.2. Реализована аутентификация и авторизация пользователей.
3.4. Настроена маршрутизация запросов к соответствующим агентам на целевых серверах и внедрено логирования операций.

Этап 4. Реализация клиентской части (фронтенд)

4.1. Создан интерфейс на базе Blazor Server с применением компонентного подхода, разработан главный макет приложения с адаптивной боковой панелью навигации.

4.2. Реализованы три основных раздела: «Управление образами», «Управление клонами», «Мониторинг» и внедрены модальные окна для подтверждения критических операций и ввода параметров.
4.3. применён корпоративный стиль оформления для обеспечения единообразия интерфейса.
4.4. Реализован механизм выбора целевого сервера для распределения нагрузки.

Этап 5. Интеграция компонентов

5.1. Обеспечено взаимодействие веб-клиента с серверной частью посредством HTTP-запросов и реализован обмен данными между сервером и агентами через REST API.
5.2. Интегрирована база данных для централизованного хранения метаданных системы.

Шаг 6. Тестирование и отладка

Проведено комплексное тестирование: Функциональное тестирование всех сценариев, тестирование производительности с большими БД, проверка обработки ошибок и отображения сообщений, тестирование безопасности. Проведено нефункциональное тестирование.

Этап 7. Документирование и подготовка к внедрению

7.1. Разработано руководство администратора по установке, настройке и обслуживанию системы.
7.2. Создано руководство пользователя с описанием сценариев работы в интерфейсе приложения.

Этап 8. Развёртывание и сопровождение

8.1. Выполнена установка компонентов на целевые серверы: веб-приложение (IIS), агенты.
8.2. Настроено резервное копирование базы метаданных и конфигурационных файлов.
8.3. Осуществлён мониторинг работоспособности системы и анализ логов в штатном режиме.

4.2 Пользовательский интерфейс

4.2.1 Описание взаимодействия с пользователем

Пользовательский интерфейс системы предназначен для специалистов IT-отдела и включает три основных раздела: управление образами, управление клонами и мониторинг. Взаимодействие строится по следующему принципу:

Авторизация и навигация

После входа по логину/паролю пользователь попадает на главную страницу — «Управление образами». Навигация осуществляется через боковое меню (три раздела) и выпадающий список выбора целевого SQL-сервера в верхней панели.

Основные сценарии работы

- Создание образа:

Пользователь выбирает сервер, переходит в раздел «Образы», нажимает кнопку «Создать образ», заполняет форму (указывает тип источника, имя образа и пути к файлам), подтверждает действие. Система отображает индикатор выполнения, после завершения операции показывает результат: «УСПЕХ» или «ОШИБКА» с описанием.

- Создание клона:

Пользователь выбирает сервер, переходит в раздел «Клоны», нажимает кнопку «Создать клон», выбирает существующий образ из списка, указывает имя клона и путь к VHD-файлу, подтверждает действие. Система выполняет операцию и отображает результат.

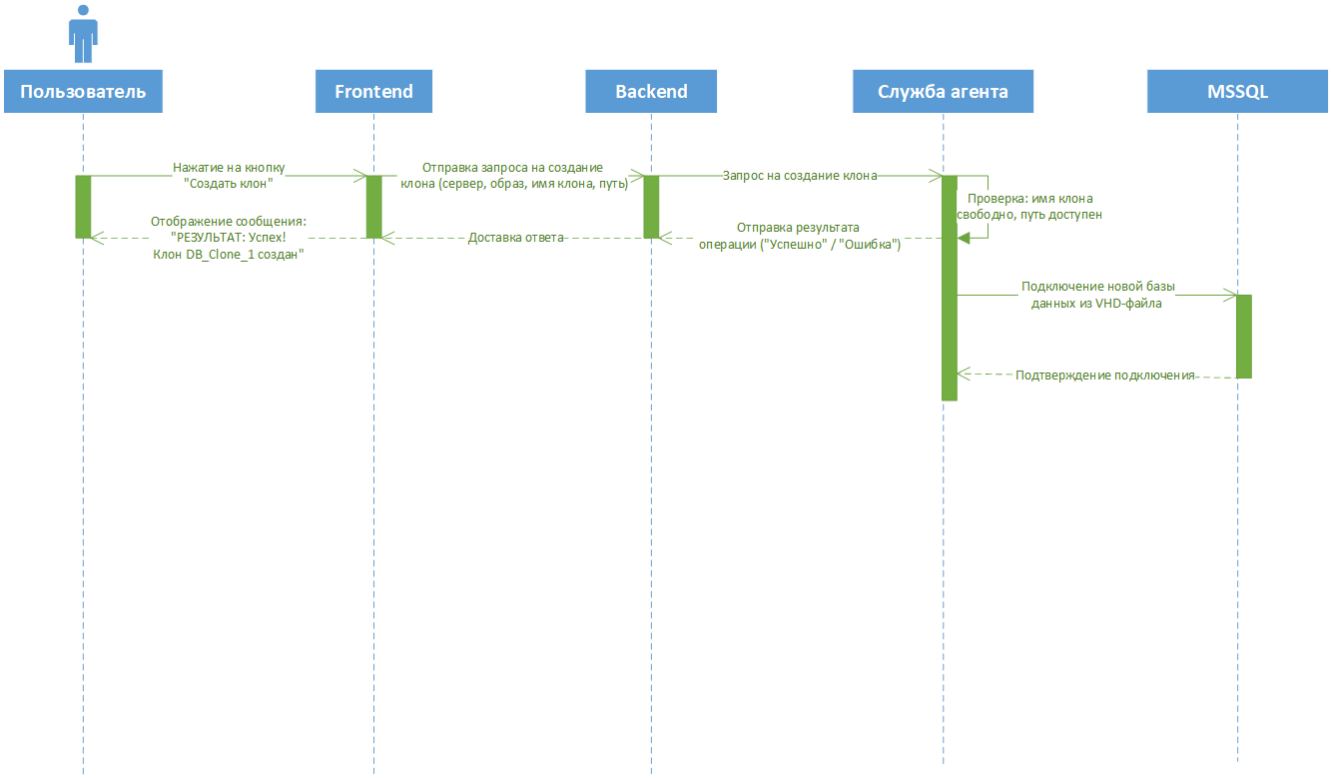
- Удаление объекта:

Пользователь выбирает объект в таблице и нажимает кнопку «Удалить». Система запрашивает подтверждение во всплывающем окне. После подтверждения выполняется операция удаления. При удалении образа система автоматически проверяет наличие зависимых клонов и блокирует операцию, если такие клоны существуют.

- Мониторинг:

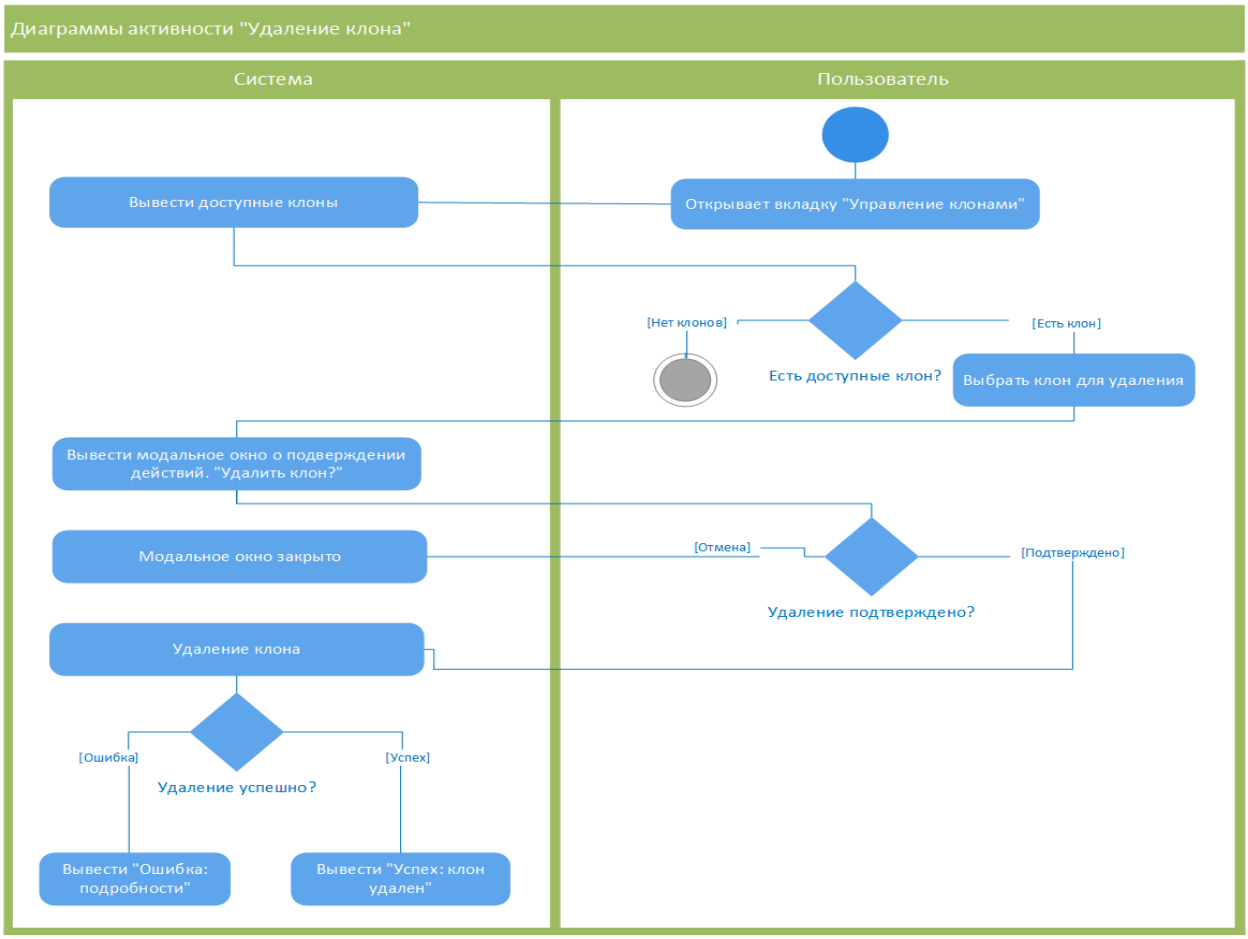
Пользователь выбирает сервер в разделе «Мониторинг». Система автоматически загружает данные с агента и отображает две таблицы: «Образы» и «Клоны» с ключевыми атрибутами (имя, путь к VHD, размер, дата создания).

На рисунке 4 представлена UML-диаграмма последовательности для сценария «Создание клона», отражающая взаимодействие четырёх участников: Пользователь - Веб-клиент – Бэкенд - Агент.



Изображение 4 - UML-диаграмма последовательности использования

На рисунке 5 представлена UML-диаграмма активности для сценария «Удаление образа», показывающая логику принятия решений (проверка зависимых клонов) и два возможных пути выполнения (успех/ошибка).



Изображение 5 - UML-диаграмм активности.

Такой подход обеспечивает интуитивно понятное взаимодействие, минимизацию ошибок и соответствие требованиям корпоративного стандарта.

4.2.2 Определение операций пользователей и составление функциональных блоков

На основе анализа прецедентов использования системы были определены следующие операции, которые пользователь может выполнять в рамках работы с приложением:

1. Авторизация в системе: ввод логина и пароля; проверка учётных данных, переход на главную страницу при успехе.
2. Выбор целевого SQL-сервера из выпадающего списка; обновление контекста работы, загрузка данных с выбранного сервера.
3. Переход в раздел «Управление образами»; отображение таблицы с существующими образами и кнопки «Создать образ».
4. Переход в раздел «Управление клонами»; отображение таблицы с существующими клонами и кнопки «Создать клон».
5. Переход в раздел «Мониторинг»; автоматическая загрузка и отображение данных об образах и клонах на выбранном сервере.
6. Нажатие кнопки «Создать образ»; открытие модального окна с формой создания образа, включающей выбор типа источника («Из существующей БД» или «Из .bak-файла»).
7. Ввод данных в поля формы (имя образа, путь к файлу-источнику, путь сохранения VHD-файла); сохранение значений в полях формы.
8. Нажатие кнопки «Создать» в модальном окне создания образа; отправка запроса на сервер, отображение индикатора выполнения, закрытие модального окна, отображение результата операции.
9. Нажатие кнопки «Создать клон»; открытие модального окна с формой создания клона.
10. Выбор образа из выпадающего списка в форме создания клона; сохранение выбранного значения.
11. Ввод имени клона и пути к VHD-файлу клона; сохранение значений в полях формы.
12. Нажатие кнопки «Создать» в модальном окне создания клона; отправка запроса на сервер, отображение индикатора выполнения, закрытие модального окна, отображение результата операции.

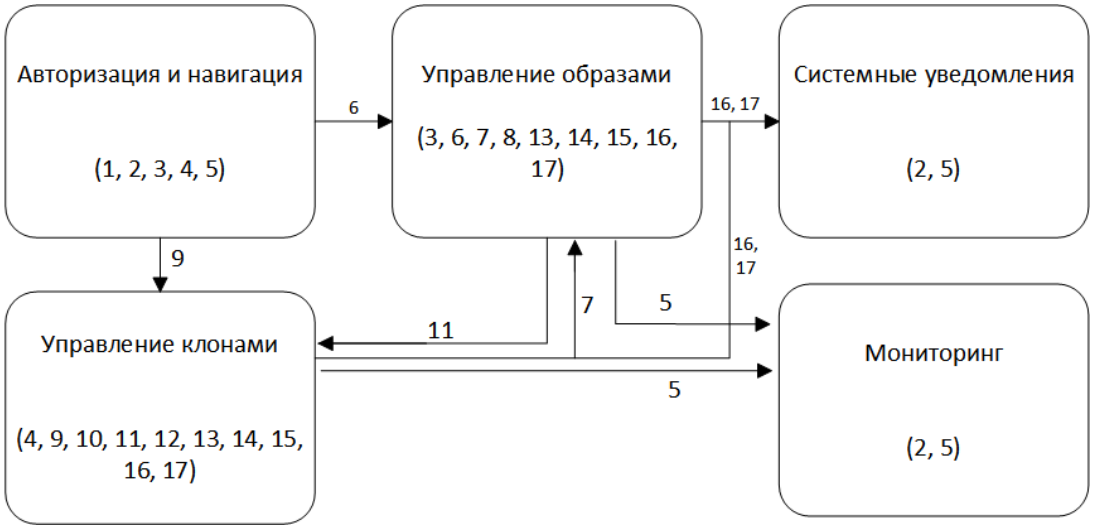
13. Нажатие кнопки «Удалить» напротив образа или клона в таблице; отображение всплывающего окна подтверждения.
14. Подтверждение удаления в всплывающем окне (кнопка «ОК»); отправка запроса на удаление, отображение результата операции, обновление таблицы.
15. Отмена операции (закрытие окна); закрытие окна, возврат к предыдущему состоянию.
16. Просмотр результата операции в диалоговом окне; отображение сообщения «УСПЕХ» или «ОШИБКА» с описанием.
17. Закрытие диалогового окна с результатом (кнопка «ОК»); закрытие окна, обновление данных при необходимости.

Функциональные блоки

- Функциональный блок 1: Авторизация и навигация
  Включает операции 1, 2, 3, 4, 5. Расположен в верхней панели (выбор сервера) и боковой панели навигации (меню разделов). Назначение: обеспечение доступа к системе и навигации между основными разделами.
- Функциональный блок 2: Управление образами
  Включает операции 3, 6, 7, 8, 13, 14, 15, 16, 17. Расположен на странице «Управление образами» с модальным окном создания и всплывающим окном подтверждения. Назначение: создание неизменяемых шаблонов баз данных для последующего клонирования.
- Функциональный блок 3: Управление клонами
  Включает операции 4, 9, 10, 11, 12, 13, 14, 15, 16, 17. Расположен на странице «Управление клонами» с модальным окном создания и всплывающим окном подтверждения. Назначение: развёртывание рабочих копий баз данных на основе существующих образов.
- Функциональный блок 4: Мониторинг
  Включает операции 2, 5. Расположен на странице «Мониторинг». Назначение: отображение актуальной информации об образах и клонах на выбранном сервере.
- Функциональный блок 5: Системные уведомления
  Включает операции 16, 17. Расположен в виде диалоговых окон, появляющихся поверх основного интерфейса. Назначение: информирование пользователя о результатах операций и запрос подтверждения критических действий.

4.2.3 Проектирование структуры экранов и схемы навигации

На основе описанных сценариев работы были выделены функциональные блоки, они представлены в схеме навигационной системы.



Изображение 6 – Схема навигации

В данной схеме **5 функциональных блоков** и **три основные экранные формы**:

- Управление образами;
- Управление клонами;
- Мониторинг.

Данные функциональные блоки соответствуют работе пользователей с информацией:

- по образам;
- по клонам;
- по состоянию системы.

Цифрами на рисунке указаны отдельные операции, выполняемые пользователями.

4.2.4 Разработка дизайна интерфейса

Дизайн пользовательского интерфейса разработан с учётом требований корпоративного стиля» и ориентирован на удобство работы специалистов IT-отдела. Интерфейс выполнен в минималистичном стиле с акцентом на функциональность и читаемость данных.

Цветовая схема и шрифт

Основная цветовая палитра соответствует корпоративному бренду компании:

Основной цвет: синий (#0c54a0) — используется для заголовков, активных кнопок и выделения важных элементов. Акцентный цвет: оранжевый (#ef7f1a) — применяется для кнопок действий («Создать образ», «Создать клон»). Фон: светло-серый (#f8f9fa) для основной области, белый (#ffffff) для таблиц и карточек. Текст черный.

Для интерфейса используется шрифт Segoe UI (системный шрифт Windows) с. Размеры шрифтов: Заголовки разделов: 24 px, полужирный, заголовки таблиц: 16 px, полужирный. Основной текст: 14 px

Межстрочный интервал установлен в 1.5 для улучшения читаемости.

Структура экранов

Все экраны приложения имеют единую структуру, состоящую из трёх основных зон:

- Верхняя панель

Содержит логотип компании, заголовок текущего раздела и выпадающий список выбора целевого SQL-сервера. Высота панели — 60 пикселей. Список серверов отображает доступные для управления серверы с индикацией статуса (онлайн/оффлайн).

- Боковая панель навигации

Расположена слева, ширина — 240 пикселей. Содержит три пункта меню:

Управление образами, Управление клонами, Мониторинг. Активный раздел выделяется синей полосой слева и полужирным шрифтом. При наведении курсора пункты меню подсвечиваются светло-серым фоном.

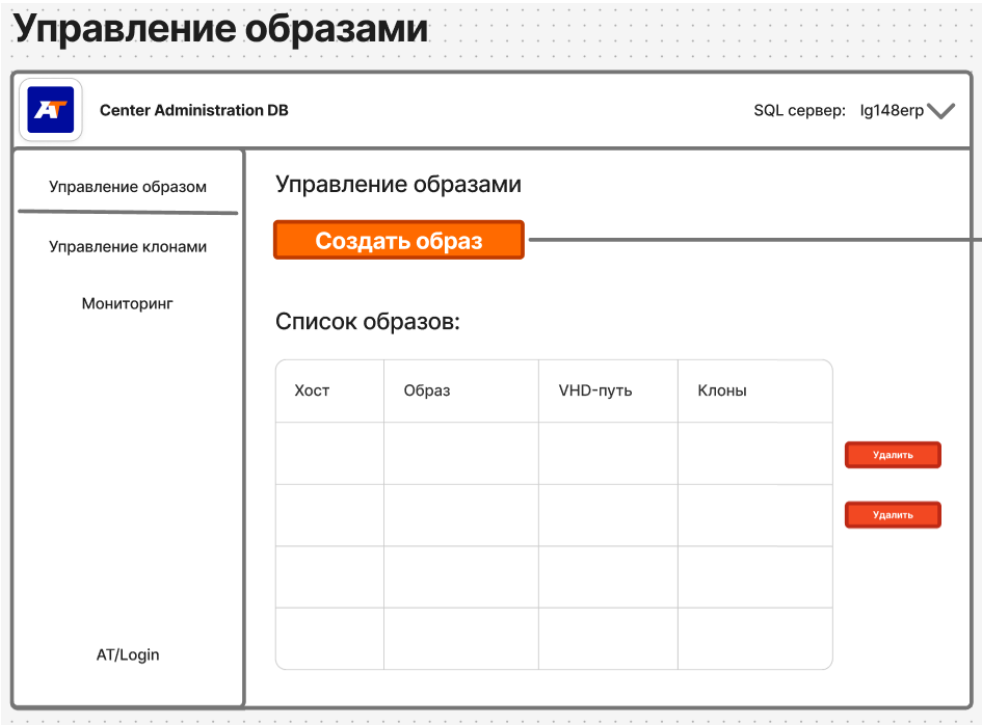
- Основная рабочая область

Занимает оставшееся пространство экрана. Содержит: Заголовок раздела, Кнопки действий («Создать образ», «Создать клон») в правом верхнем углу, Таблицу с данными (образы, клоны или информация для мониторинга), Модальные окна для форм ввода данных (появляются поверх основного интерфейса).

Макеты основных экранов

- Экран «Управление образами»

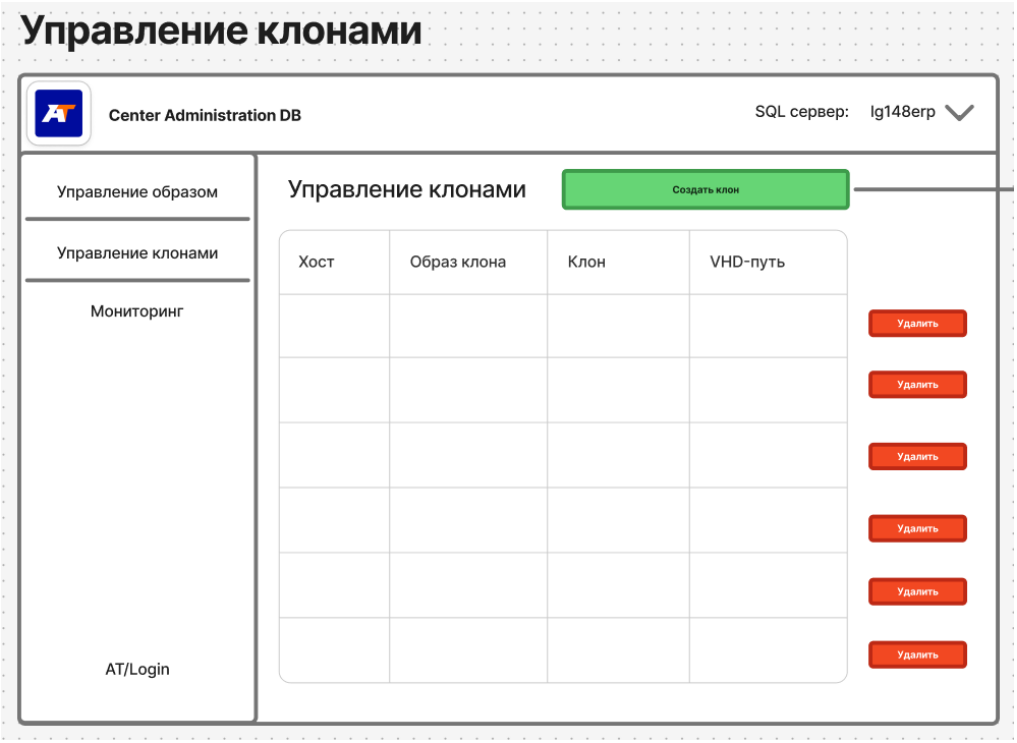
В верхней части расположена кнопка «Создать образ» оранжевого цвета. Ниже следует таблица со столбцами: «Имя образа», «Исходная БД», «Путь к VHD», «Размер», «Дата создания», для каждой записи размещена кнопка «Удалить» красного цвета. Разработанный макет экрана представлен на изображении 7.



Изображение 7 – макет страницы «Управление образами»

- Экран «Управление клонами».

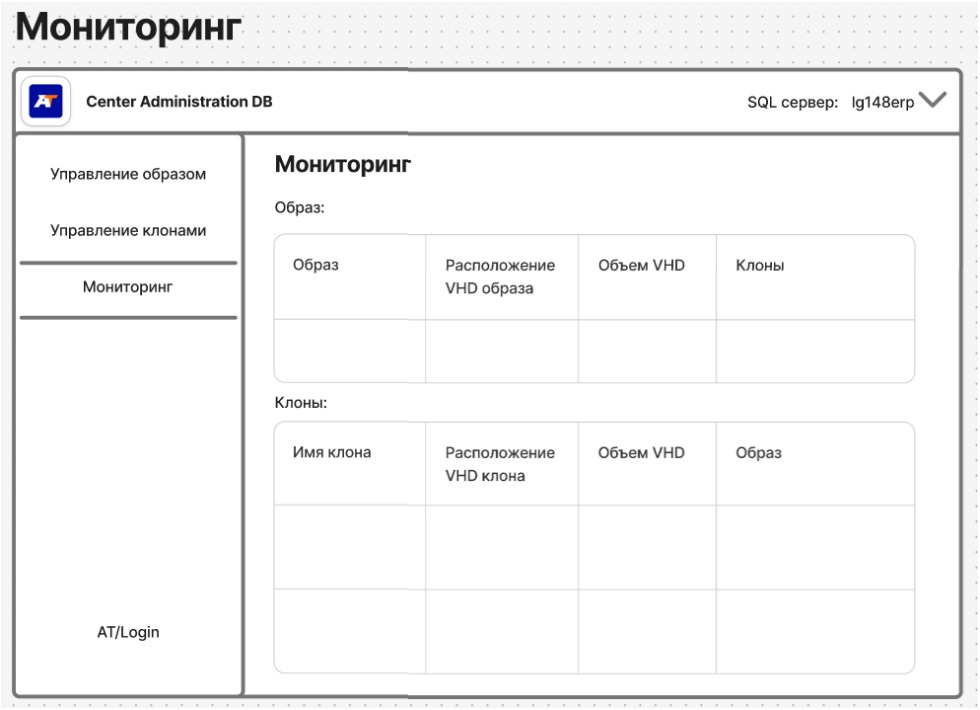
  Структура аналогична экрану образов. Разработанный макет экрана представлен на изображении 8.



Изображение 8 – макет страницы «Управление клонами»

- Экран «Мониторинг»

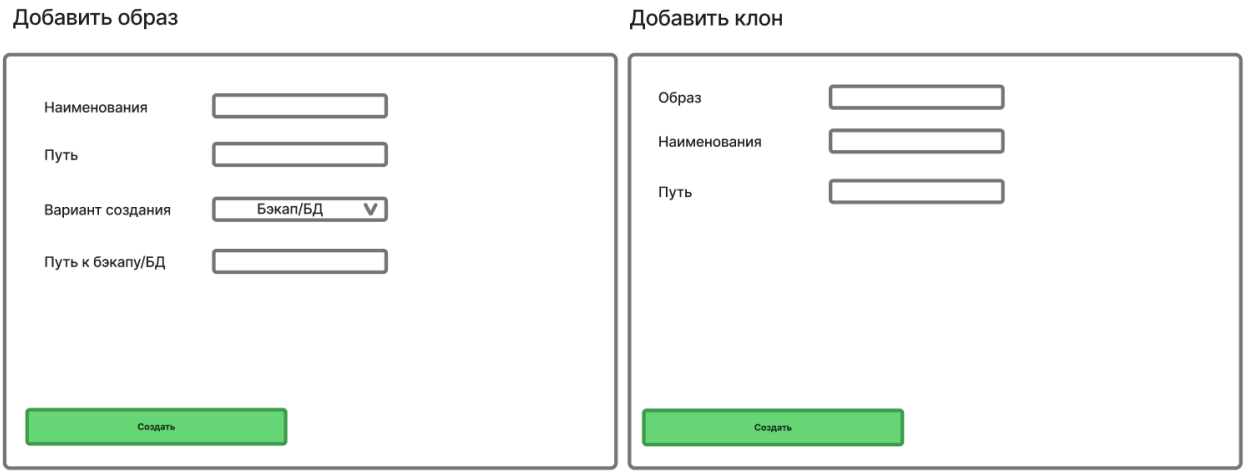
Содержит две таблицы, расположенные вертикально: Верхняя таблица «Образы» — отображает все доступные образы на выбранном сервере Нижняя таблица «Клоны» — отображает все активные клоны с указанием зависимости от образов. Между таблицами расположен разделитель с заголовком. Обновление данных происходит автоматически при выборе сервера. Разработанный макет экрана представлен на изображении 9.



Изображение 9 – макет страницы «Мониторинг»

Модальные окна

Модальные окна для создания образов и клонов имеют единую структуру: Заголовок окна по центру верхней части, форма ввода с полями (текстовые поля, выпадающие списки). Кнопка «Создать» (оранжевая) в правом нижнем углу.
При выполнении длительных операций поверх модального окна отображается индикатор загрузки в виде вращающегося круга с надписью «Выполняется операция...».
Макеты разработанных модальных окон представлен на изображении 10.



Изображение 10 – макет модальных окон «Добавить образ» и «Добавить клон»

Разработанный дизайн обеспечивает единообразие всех экранов, интуитивную навигацию и быстрое восприятие ключевой информации. Минималистичный подход без избыточной графики позволяет специалистам сосредоточиться на выполнении рабочих задач без визуальных отвлекающих факторов. Цветовое кодирование и чёткая структура элементов интерфейса соответствуют современным стандартам проектирования корпоративных приложений.

4.3 Входные, выходные и промежуточные данные

Разработанное веб-приложение работает с тремя типами данных: входными (поступающими в систему), выходными (формируемыми системой) и промежуточными (используемыми для внутренней обработки).

Входные данные поступают в систему из следующих источников:

1. **Конфигурационные файлы**:

Агент использует файл в формате JSON, содержащий параметры подключения к базе данных, пути к каталогам хранения VHD-файлов и настройки сервера. Бэкенд использует файл с параметрами подключения к центральной базе данных и настройками безопасности.

2. **Данные от пользователя**

Через веб-интерфейс пользователь вводит: имя образа или клона (текстовая строка до 255 символов), путь к исходной базе данных или резервной копии (.bak), путь сохранения VHD-файла, выбор целевого SQL-сервера из списка. Все данные передаются в формате JSON через HTTP-запросы к серверной части.

3. **Данные от агентов**

Агенты, развёрнутые на SQL-серверах, передают в центральную систему:

- информацию о дисковом пространстве,
- список существующих образов и клонов,
- результаты выполнения операций (успех/ошибка с описанием).

Данные передаются в формате JSON через защищённое HTTPS-соединение.

Выходные данные формируются системой для пользователя и других компонентов:

1. **Данные для веб-интерфейса**

Серверная часть возвращает данные в формате JSON:

- списки образов и клонов с атрибутами (имя, путь к VHD, размер, дата создания), результаты операций (статус выполнения, сообщения об ошибках).

Браузер отображает эти данные в виде трёх основных страниц: «Управление образами», «Управление клонами», «Мониторинг».

2. **Логи системы**

Агенты и серверная часть ведут текстовые логи. Логи хранятся в каталогах приложения и используются для диагностики и аудита операций.

3. **Промежуточные данные**

Промежуточные данные используются внутри системы для хранения состояния и обеспечения работы компонентов:

4. **Данные в базе данных**

Центральная база данных SqlCloneConfig содержит две таблицы: Images — информация об образах и Clones — информация о клонах.

5. **Временные файлы**

При выполнении операций клонирования система создаёт временные VHD-файлы в каталоге. После завершения операции файлы перемещаются или удаляются.

Такая структура данных обеспечивает надёжное хранение информации, минимизирует количество обращений к диску и сети, а также позволяет легко восстанавливать состояние системы после перезапуска компонентов. Все данные передаются в стандартизированных форматах (JSON, текст), что упрощает интеграцию и отладку системы.

4.4 Реализация используемых методов хранения, обработки и передачи информации об объектах предметной области

4.4.1 Методы хранения данных

В рамках реализации программного средства для управления копиями баз данных применены методы хранения информации, обоснованные в разделе 2.1.3.1. Для обеспечения надёжности, целостности и масштабируемости данных выбрана реляционная модель хранения. В качестве системы управления базами данных (СУБД) используется Microsoft SQL Server Management Studio, что обусловлено поддержкой транзакций и возможностью аудита операций.

Информационная система предполагает гибридный подход к хранению данных: метаданные об объектах предметной области (образах и клонах) хранятся в таблицах реляционной базы данных, а бинарные данные (файлы виртуальных жёстких дисков формата VHD) размещаются в файловой системе сервера.

1. Таблица Images

- Имя таблицы: Images

- Назначение: Таблица предназначена для хранения метаданных об исходных образах баз данных. Она содержит информацию, необходимую для идентификации образа, локализации его файла в файловой системе, а также аудита операций создания и удаления. Описание полей таблицы представлено ниже:

Таблица 5 – Структура таблицы Images

Имя поля	Тип данных	Назначение	Ключ
id	число	Уникальный идентификатор образа	Первичный ключ (PK)
name	текст	Имя образа	
pathVHD	текст	Полный путь к файлу VHD образа	
sourceDbName	текст	Имя исходной базы данных	
fileNameVHD	текст	Имя файла VHD	
diskPath	текст	Корневой путь к каталогу хранения	
createdDate	дата	Дата и время создания записи	
deletedDate	дата	Дата и время удаления	

2. Таблица Clones

- Имя таблицы: Clones
- Назначение: Таблица предназначена для хранения сведений о созданных клонах баз данных. Она фиксирует параметры клона, путь к его файлу и ссылку на родительский образ, обеспечивая отслеживание зависимостей между объектами. Описание полей таблицы представлено ниже:

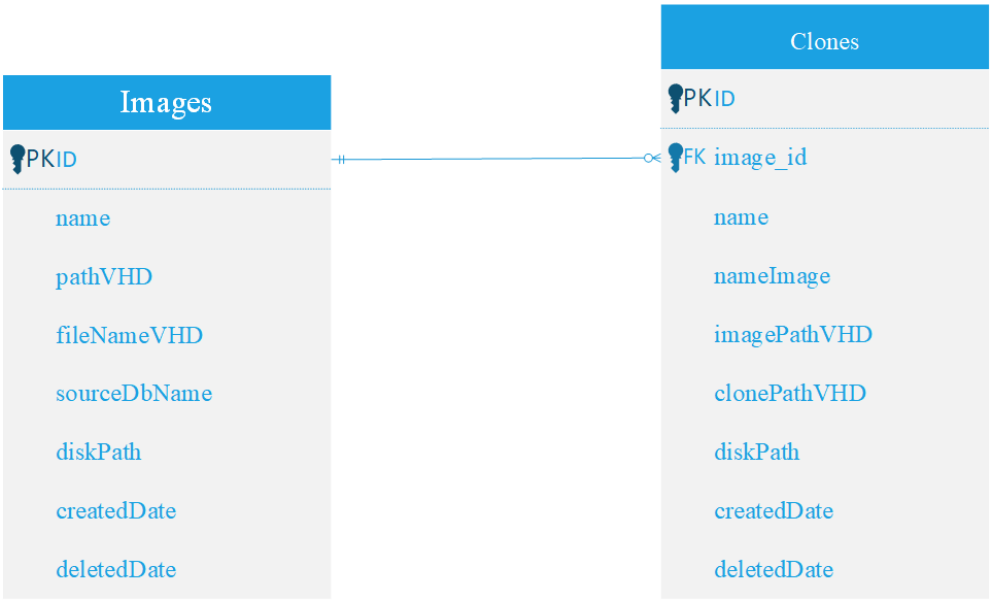
Таблица 6 – Структура таблицы Clones

Имя поля	Тип данных	Назначение	Ключ
id	число	Уникальный идентификатор клона	Первичный ключ (PK)
name	текст	Имя клона	
nameImage	текст	Имя образа	
image_id	число	Ссылка на исходный образ	Внешний ключ (FK) к Images
imagePathVHD	текст	Путь к исходному VHD-образу	
clonePathVHD	текст	Путь к файлу VHD клона	
diskPath	текст	Корневой путь к каталогу хранения клона	
createdDate	дата	Дата и время создания записи	
deletedDate	дата	Дата и время удаления	

Связь между таблицами реализована по принципу «один ко многим»: один образ (Images) может являться источником для множества клонов (Clones). Целостность ссылочных данных обеспечивается на уровне СУБД посредством внешнего ключа в таблице Clones.

Файлы виртуальных дисков (VHD), пути к которым хранятся в полях, физически размещаются в каталогах файловой системы сервера, указанных в поле diskPath. Физическое размещение файлов организовано таким образом, чтобы обеспечить изоляцию данных различных образов и клонов, а также упростить процедуры резервного копирования и восстановления.

Схема связей между таблицами базы данных представлена на Изображении 11.



Изображение 11 – ER-диаграмма связей между таблицами

4.4.2 Алгоритмы реализации используемых математических моделей

В рамках данной работы разработка новых математических моделей не предусмотрена. Обработка информации базируется на логических правилах и ограничениях предметной области, описанных в пункте 2.1.3.2. Реализация данных правил выполнена в виде алгоритмов контроля целостности и безопасности операций.

Ниже приведено описание алгоритмов, реализующих ключевые ограничения системы.

1. Алгоритм проверки уникальности имени клона

Алгоритм обеспечивает выполнение правила уникальности имён в пределах сервера. Последовательность действий:

1. Получение входных данных: запрошенное имя клона.
2. Формирование запроса к базе данных для выборки имён всех активных записей из таблиц Images и Clones.
3. Сравнение с полученным множеством имён.
4. Принятие решения:
    - Если совпадение найдено: отмена операции, возврат ошибки.
    - Если совпадений нет: передача управления модулю создания клона.
5. Завершение алгоритма.

2. Алгоритм проверки зависимости образа при удалении

Алгоритм реализует правило запрета удаления образа при наличии зависимых клонов.

Последовательность действий:

1. Получение входных данных: идентификатор образа, подлежащего удалению.
2. Выполнение выборки из таблицы Clones по условию image_id = id_image.
3. Анализ количества полученных записей:
    - Если счетчик записей > 0: блокировка удаления, формирование уведомления о наличии зависимых объектов.
    - Если счетчик записей = 0: разрешение на удаление записи из таблицы Images.
4. Завершение алгоритма.

3. Алгоритм контроля дискового пространства

Алгоритм обеспечивает выполнение условия достаточности ресурсов перед записью данных.

Последовательность действий:

1. Получение входных данных: требуемый размер файла.
2. Запрос к операционной системе о объеме свободного места на целевом диске.
3. Сравнение значений: Свободного места ≥ size_req.
4. Принятие решения:
    - При истинности условия: инициирование процесса создания файла.
    - При ложности условия: прерывание операции, генерация исключения «Недостаточно дискового пространства».
5. Завершение алгоритма.

Описанные алгоритмы реализованы в коде агента управления в виде последовательности условных операторов и обеспечивают корректность функционирования системы согласно требованиям предметной области.

Изображение 12 – UML диаграмма активности (Обязательно в 4.4.3 или можно пропустить?)

4.4.4 Алгоритмы применения методов графического анализа данных

В рамках реализации программного средства для управления копиями баз данных применён алгоритм визуального представления информации, ориентированный на задачи оперативного управления и контроля. Учитывая операционно-административный характер предметной области, приоритет отдан табличным и текстовым формам отображения данных, обеспечивающим высокую плотность информации и удобство её восприятия администратором.

Алгоритм визуализации данных в системе

Шаг 1. Выбор данных для отображения

Для представления пользователю отбираются атрибуты сущностей, имеющие оперативную значимость:

- для сущности «Образ» (Image): имя, путь к файлу, дата создания;
- для сущности «Клон» (Clone): имя, исходный образ, путь к VHD-файлу, дата создания. Данные извлекаются из реляционной базы данных посредством выборки полей таблиц Images и Clones и передаются на клиентскую сторону в формате JSON.

Шаг 2. Применение способов визуализации.

В системе реализованы следующие предпочтительные методы представления информации:

1. **Табличное отображение**: основные сущности выводятся в виде структурированных таблиц с возможность. Данный способ выбран ввиду его эффективности для сравнения атрибутов множества объектов.
2. **Диалоговые окна**: используются для подтверждения критических действий (удаление образа, удаление клона) и отображения результатов выполнения операций. Это обеспечивает защиту от ошибочных действий пользователя и повышает информативность интерфейса.

Шаг 3. Интерпретация и формирование выводов

На основе представленной визуальной информации пользователь (администратор) формирует управленческие решения:

- оценка текущего состояния хранилища образов и клонов;
- идентификация объектов, требующих внимания (ошибки, зависшие операции);
- принятие решений о создании новых клонов или удалении устаревших образов.

Сложные графические формы (диаграммы, графики, тепловые карты) в системе не применяются, так как они не соответствуют характеру решаемых задач и не повышают эффективность принятия решений в контексте управления копиями баз данных.

4.4.5 Алгоритмы использования технологий передачи данных

Передача данных реализована в соответствии с трёхзвенной архитектурой, обеспечивающей модульность, масштабируемость и безопасность взаимодействия компонентов. Обмен информацией осуществляется между веб-клиентом, серверной частью (бэкендом) и агентами, развёрнутыми на целевых серверах баз данных.

1. Межмодульная передача данных внутри системы

Взаимодействие Веб-клиента и бэкенда

Протокол: передача данных осуществляется посредством HTTP/HTTPS-запросов к контроллерам ASP.NET Core.

- **Формат данных**: все запросы и ответы сериализуются в формате JSON, что обеспечивает лёгкость парсинга.
- **Алгоритм обмена**:
    1. Клиентская часть (Blazor WebAssembly) формирует запрос к соответствующему API (например, GET /api/images).
    2. Бэкенд обрабатывает запрос, при необходимости обращаясь к базе.
    3. Результат операции или запрошенные данные возвращаются клиенту в виде JSON-объекта.
    4. Клиент десериализует ответ и обновляет пользовательский интерфейс.

Взаимодействие Бэкенд и Агент

- **Протокол**: взаимодействие организовано через REST API, где бэкенд выступает в роли инициатора запроса, а агент — в роли получателя команд.
- **Механизм опроса**: бэкенд периодически опрашивает агентов посредством запросов GET /api/agent/status для получения информации о текущем состоянии.

- **Передача команд**: при выполнении операций управления (создание клона, удаление образа) бэкенд отправляет агенту команду, содержащую необходимые параметры (пути к файлам, имена баз данных).
- **Алгоритм обработки**:
    1. Агент получает HTTP-запрос от бэкенда.
    2. Агент выполняет запрошенную операцию (обращение к SQL Server или файловой системе).
    3. Результат выполнения (успех/ошибка) возвращается бэкенду для обновления метаданных и информирования пользователя.

2. Взаимодействие с внешними источниками и потребителями данных

Взаимодействие Агент и СУБД Microsoft SQL Server

- **Технология**: агент взаимодействует с экземпляром SQL Server посредством выполнения T-SQL-запросов (SELECT, RESTORE DATABASE, CREATE DATABASE).
- **Назначение**: обеспечение операций клонирования, проверки состояния баз данных и получения метаданных на уровне СУБД.

Взаимодействие Агент и Файловая система

- **Технология**: для работы с файлами виртуальных дисков (VHD) используются стандартные методы платформы .NET.
- **Назначение**: физическое создание, копирование и удаление файлов образов и клонов в структуре каталогов сервера.

3. Обеспечение безопасности передачи данных

Все каналы связи между компонентами системы защищены посредством протокола HTTPS, что обеспечивает:

- аутентификацию сторон взаимодействия;
- целостность передаваемой информации.

4. Программно-алгоритмическое обеспечение связи «клиент-сервер»

- На стороне бэкенда реализована маршрутизация запросов через контроллеры ASP.NET Core с применением атрибутов [HttpGet], [HttpPost].
- Для сериализации/десериализации JSON используется встроенный инструменты библиотек C#.

Таким образом, разработанная технология передачи данных обеспечивает надёжный, безопасный и стандартизированный обмен информацией между всеми компонентами системы, соответствующий требованиям предметной области и современным практикам разработки распределённых приложений.

4.5 Описание архитектурного решения

4.5.1 Структурная организация программной системы

Разработанное программное средство для управления копиями баз данных Microsoft SQL Server посредством VHD-образов построена на основе трёхзвенной архитектуры. Структурная организация системы предусматривает выделение логических слоёв, каждый из которых имеет определённый набор ответственности и взаимодействует с другими слоями через чётко определённые интерфейсы.

Организация структурных элементов. Программная система декомпозируется на следующие основные компоненты:

Компонент «Веб-клиент»

- Назначение: обеспечение пользовательского интерфейса для взаимодействия администратора с системой.
- Технологическая реализация: Blazor Server.
- Функциональная ответственность: отображение списков образов и клонов, формирование запросов на выполнение операций, визуализация статусов и результатов, обработка пользовательского ввода.
- Интерфейсы взаимодействия: отправляет HTTP/HTTPS-запросы к бэкенду, принимает и десериализует ответы в формате JSON.

Компонент «Серверная часть» (Application Layer / Backend)

- Назначение: координация бизнес-логики, обработка запросов клиентов, управление взаимодействием с агентами и базой метаданных.
- Технологическая реализация: ASP.NET Core Web API.
- Функциональная ответственность: маршрутизация входящих запросов от веб-клиента; валидация входных данных и применение бизнес-правил; управление сессиями, формирование команд для агентов и обработка их ответов;
- Интерфейсы взаимодействия: принимает REST-запросы от клиента, инициирует HTTP-вызовы к агентам, выполняет оопераций с базой данных.

Компонент «Агент»

- Назначение: выполнение низкоуровневых операций на целевых серверах баз данных.
- Технологическая реализация: на платформе .NET.
- Функциональная ответственность: выполнение T-SQL-запросов к экземплярам Microsoft SQL Server; управление файлами виртуальных дисков (VHD) в файловой системе; мониторинг состояния локальных ресурсов и отправка отчётов бэкенду.
- Интерфейсы взаимодействия: принимает команды от бэкенда через REST API, взаимодействует с СУБД и файловой системой с помощью встроенных инструментов .NET

Компонент «База данных»

- Назначение: централизованное хранение структурированной информации об образах, клонах и истории операций.
- Технологическая реализация: реляционная СУБД (Microsoft SQL Server).
- Функциональная ответственность: обеспечение целостности и аудита данных.
- Интерфейсы взаимодействия: принимает запросы от бэкенда через провайдера .NET.

Объединение компонентов в подсистемы. Компоненты системы группируются в две логические подсистемы:

Подсистема управления: включает веб-клиент и серверную часть. Отвечает за предоставление интерфейса администратору, обработку бизнес-логики и координацию распределённых операций.

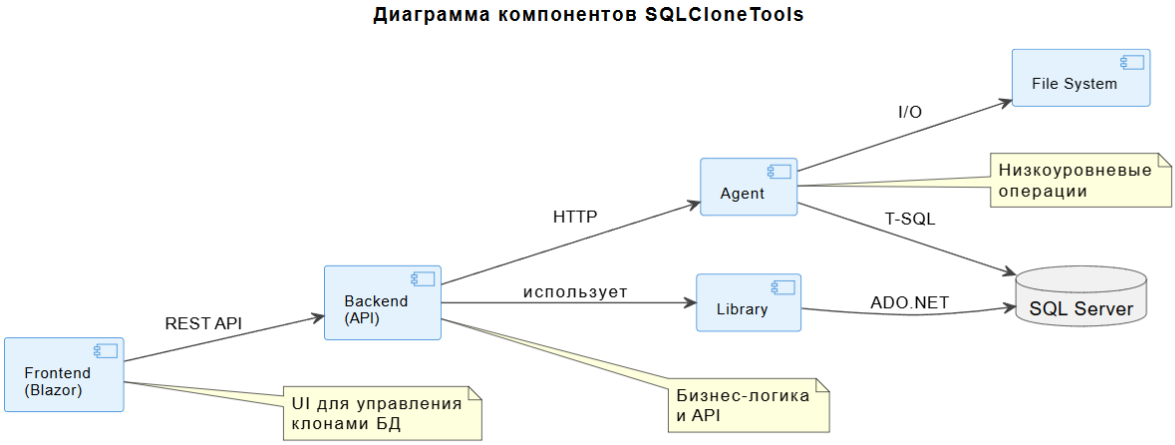
Подсистема исполнения: включает агенты и целевые серверы баз данных. Отвечает за физическое выполнение операций клонирования, управления дисковым пространством и взаимодействия с СУБД.

Место системы в более крупной инфраструктуре

Разрабатываемая программная система функционирует в составе корпоративной ИТ-инфраструктуры организации, интегрируясь с существующими серверами баз данных Microsoft SQL Server. Система не заменяет штатные средства администрирования СУБД, а дополняет их, предоставляя специализированный инструмент для автоматизации операций клонирования и управления копиями баз данных.

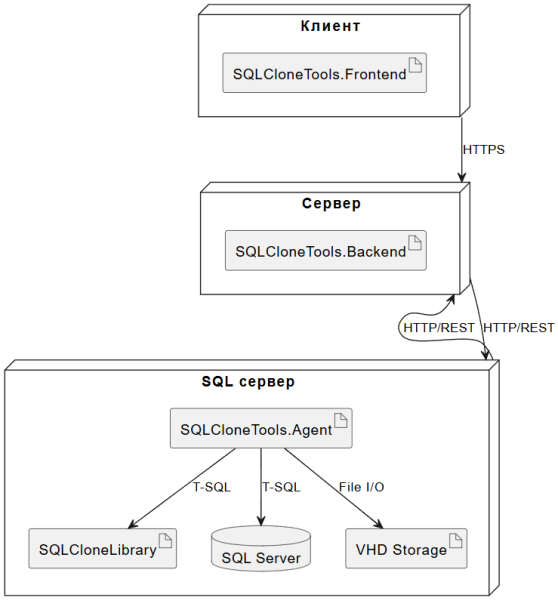
Статические структурные представления

Для иллюстрации архитектурных зависимостей между компонентами использована диаграмма компонентов UML (Изображение 13). Диаграмма демонстрирует интерфейсы, предоставляемые и потребляемые каждым компонентом, а также направления зависимостей, что позволяет оценить связность и модульность системы.



Изображение 13 – UML диаграмма компонентов

Физическая организация системы представлена на диаграмме развёртывания UML (Изображение 14). Диаграмма отображает распределение программных артефактов по аппаратным узлам: серверу приложений, серверам баз данных и рабочим станциям администраторов.



Изображение 14 – UML диаграмма развёртывания

Таким образом, структурная организация программной системы обеспечивает чёткое разделение ответственности, слабую связность компонентов и возможность гибкого масштабирования при увеличении количества управляемых серверов баз данных.

4.5.2 Архитектура программного кода

Структурная организация кода по компонентам системы/ Исходный код программной системы разделён на четыре основных проекта (решение Visual Studio / решения .NET), соответствующих компонентам архитектуры:

SQLCloneTools.Frontend — проект клиентской части (Blazor).

SQLCloneTools.Backend— проект серверной части (ASP.NET Core Web API).

SQLCloneTools.Agent. — проект агента (Консольное приложение с HTTP сервером).

SQLCloneLibrary— общая библиотека классов, используемая агентом (модели, интерфейсы).

1. Описание функциональных блоков

1.1. Общая библиотека (SQLCloneLibrary)

Обеспечивает согласованность типов данных и контрактов взаимодействия между компонентами.

Таблица 7 – Структура библиотеки

Блок	Ключевые элементы (классы)	Назначение
Models	Message.cs	Транспортные объекты для передачи данных об результатах операций.
DataAccess	DatabaseController.cs, PowerShellScript	Операции с БД, выполнение T-SQL запросов, автоматизация системных операций.
Services	ICloneService, CloneService, IVHDService, VHDService	Интерфейсы и реализации бизнес-логики: валидация, управление транзакциями, работа с VHD.

2.2. Агент (VhdCloneManager.Agent)

Агент реализует низкоуровневые операции на целевых серверах. Код агента структурирован по принципу обработки команд и выполнения специализированных задач.

Таблица 8 – Структура агента

Блок	Назначение
AgentHostedService	Основной цикл службы: приём запросов, управление жизненным циклом.

CommandHandler	Диспетчер команд: маршрутизация входящих запросов к обработчикам.
VhdFileManager	Интерфейсы и реализации бизнес-логики: валидация, управление транзакциями, работа с VHD.
SqlDatabaseManager	Управление файлами виртуальных дисков через System.IO (копирование, удаление).

2.3. Серверная часть (SQLCloneTools.Backend)

Серверная часть реализует бизнес-логику, координацию компонентов и предоставление API. Реализует бизнес-логику и API, следует паттерну «Контроллер–Сервис».

Таблица 9 – Структура серверной части

Блок	Назначение
ImagesController, ClonesController	Обработка HTTP-запросов клиентов, маршрутизация по путям /api/images, /api/clones.
AgentsController	Управление агентами: мониторинг статуса, отправка команд.
Service Registration	Регистрация зависимостей (ICloneService, IVHDService) в контейнере DI.

2.4. Клиентская часть (VhdCloneManager.Client)

Предоставляет пользовательский интерфейс для администрирования системы.

Таблица 10 – Структура клиентской части

Блок	Назначение
Pages/*.razor	Страницы управления образами и клонами, отображение таблиц и форм.
Services/ApiClient	Типизированный клиент для вызова методов бэкенда через HTTP.
Components/ConfirmDialog.razor	Модальный диалог подтверждения критических операций.

2. Взаимосвязь функциональных блоков

Взаимодействие между описанными блоками организовано посредством чётко определённых интерфейсов.

Основные принципы взаимосвязи:

- Контроллеры зависят от интерфейсов сервисов (ICloneService), а не от реализаций.
- Сервисы используют абстракции SQLCloneLibrary для работы с агентами и БД.
- Агент обрабатывает команды через единый CommandHandler.
- Клиент делегирует сетевые вызовы сервису.

Графическое представление структуры кода и зависимостей между ключевыми классами приведено на диаграмме классов UML (Изображение 15).

Изображение 15 – UML диаграмма классов программной системы

(вставить диаграмму классов, отражающую: иерархию контроллеров и сервисов бэкенда; структуру классов агента и их взаимодействие с внешними зависимостями; основные модели данных и их связи; направления зависимостей между слоями) В РАБОТЕ

4. Принципы организации кода

При разработке программного кода соблюдались следующие принципы:

- Единая ответственность: каждый класс решает одну задачу, что упрощает тестирование.
- Инкапсуляция деталей реализации: низкоуровневые операции (работа с файлами, SQL-запросы) скрыты за интерфейсами сервисов.
- Согласованность именования: используется соглашение PascalCase для классов и методов, camelCase для локальных переменных, что соответствует стандартам .NET.
- Документирование: все публичные типы и методы снабжены XML-комментариями для генерации документации API (Swagger) и облегчения сопровождения кода.

Таким образом, архитектура программного кода обеспечивает высокую степень модульности, позволяет изолировать изменения в пределах отдельных компонентов и поддерживает дальнейшее расширение функциональности системы без нарушения её целостности.

5 ТЕСТИРОВАНИЕ И ОПТИМИЗАЦИЯ

5.1 План тестирования

{В этом разделе приводится план тестирования ПС – контрольный список проверок, которые помогают тестировщику протестировать приложение или отдельные функции.

Необходимо указать, какие виды тестирования будут являться наиболее предпочтительными для вашего ПС. Обосновать свой выбор.

А также обосновать то, что другие виды тестирования проводиться не будут.}

Текст Текст

5.2 Результаты тестирования и оптимизация

{Описать процесс тестирования (пошагово, с подтверждением скриншотами), проведенного в соответствии с разработанным планом тестирования ПС. Оформить заключение о проведенном тестировании.

По результатам тестирования ПС может быть проведена техническая, алгоритмическая или программная оптимизация. Описать процесс оптимизации, подтверждая соответствующими скриншотами }

Текст Текст

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

{В этот раздел могут быть включены следующие пункты:

- порядок установки и настройки программного средства на компьютер пользователя или первичная настройка технического средства;

- действия пользователя в случае сбоя ПС или ПТС;

- последовательность действий пользователя для решения своих основных задач при работе с программным или техническим средством.

Следует привести образы экранов (ScreenShots) с пояснениями для лучшего восприятия раздела. Приветствуется создание помощи по установке и эксплуатации ПС или ПТС.}

Текст Текст Текст

ЗАКЛЮЧЕНИЕ

{Заключение – последовательное, логически стройное изложение полученных итогов и их соотношение с общей целью и конкретными задачами, поставленными и сформулированными во введении.

Заключение отражает оценку работы, подчеркивает актуальность и практическую её значимость, и включает рекомендации по практическому использованию ее результатов.

Заключение должно содержать краткие выводы по результатам ВКР.

Выводы формулируются по пунктам так, как они должны быть оглашены в конце доклада на защите.

- В этом разделе необходимо указать решена задача полностью или частично:

- подвести итоги проделанной работы – что сделано для решения поставленной задачи, это может быть:

- разработанное и реализованное ПС или ПТС. Следует детализировать, что было сделано, покомпонентно, в соответствии с составленным планом разработки;
- предложен новый подход (технология) к решению подобных задач;
- предложен новый метод реализации ПС или ПТС.

Следует отметить возможные точки роста (развития) ПС или ПТС.

Например:

- *добавить новые функции (блоки, режимы работы);*
- *распространить на новый класс задач, другой тип данных.*

В Заключении следует:

- привести объем созданного программного средства (в любых единицах, например, в килобайтах, строках кода, классах, процедурах, функциях ит.д.).
- - отразить апробацию работы: выступления на семинарах, публикации, выступления на конференциях и конкурсах (полученные дипломы, грамоты и т.п.), акты о внедрении, реально действующий сайт (адрес) и т.д.}

Текст Текст Текст

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

{Список опубликованных печатных, либо интернет- источников информации, используемых при написании текста ПЗ.

Сведения об источниках можно располагать в порядке появления ссылок на источники в тексте документа.

Источники необходимо нумеровать арабскими цифрами без точки и печатать с абзацного отступа.

Ссылки на использованные источники следует приводить в квадратных скобках в тексте, в конце предложения.

Примеры библиографического описания источников}

1 Численное исследование процессов переноса и трансформации газовых и аэрозольных примесей в шлейфе выбросов Норильского промышленного района / В.Ф. Рапута, В. Симоненков, Б.Д. Белан, Т.В. Ярославцева / «Оптика атмосферы и океана», 31, № 6, 2018 – С. 438-439.

2 Вострикова Л.Г. Финансовое право [Электронный ресурс] : учебник / Л.Г. Вострикова. – Электрон. текстовые дан. – М.: Равновесие: Юстинформ, 2005. – 1 электрон. опт. диск (CD). – Загл. с контейнера.

3 Исследовано в России: многопредмет. науч. журн. / Моск. физ.-техн. ин-т. – Электрон. журн. – Долгопрудный : МФТИ, 1998.– URL : http://zhurnal.mipt.rssi.ru, свободный. –Яз. рус. – (Дата обращ. ……..)

4 Мозолин В. П. Модернизация права собственности в экономическом измере-нии [Электронный ресурс] / В. П. Мозолин. – Электрон. ст. – М., 2011. – URL : http://www.norma-verlag.com/journal/2011/1, свободный. – Яз. рус. – Аналог печат. изд. (Журнал российского права. – 2011. – № 1). – (Дата обращ. 23.06.2011).

ПРИЛОЖЕНИЯ

Приложение А

{Содержит **техническое задание на реализацию программной системы**, разработанное при непосредственном участии представителей заинтересованных организаций.}

Приложение Б

{Содержит примеры входных данных в виде изображений, таблиц, фрагментов текстовых документов и.т.д.}

Приложение В

{Содержит различные схемы, диаграммы, иллюстрирующие алгоритмы работы разработанной программной системы.}

Приложение Г

{Содержит результаты тестовых испытаний на различных тестовых данных.

Например, при наличии в составе ПС расчетно-графического модуля, полученные в результате работы, иллюстрации и таблицы, по которым можно судить о корректной работе программы.}

Приложение Д

{Содержит фрагменты листинга программного кода.

Обязательно наличие поясняющего текста (для чего предназначен фрагмент кода, функция, класс)!

Привести не менее 10 страниц кода. Текстовые пояснения оформляются в соответствии с ГОСТ.

Параметры оформления листинга кода: размер шрифта 10, допускается расположение в 2 столбца, одинарный междустрочный интервал..}

Приложение Е

{При наличии, можно привести Акт о внедрении в эксплуатацию разработанной программной системы.}